

A38 - Vehicle Weigh System

Final Design Document

Team Members:

Chinmayi Achanta	Project Manager/ML
Noah Albert	Hardware-Sensor
Joseph Benno	ML
Dawson Holmes	Backend-Database
Anant Kumar	Hardware-Sensor

Sponsor Information:

Dr. Liqiang Wang

Dr. Zihang Zou

Scott Piersal

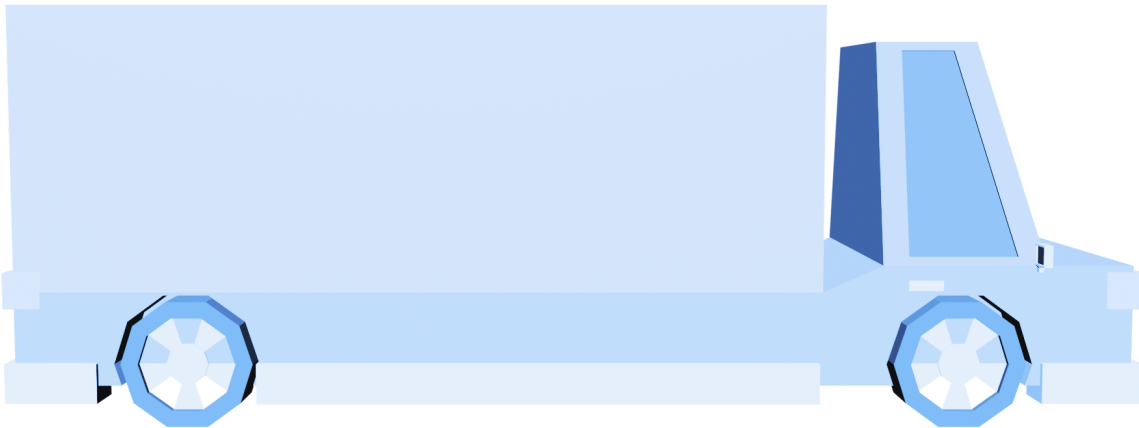


Table of Contents

1. Executive Summary	6
1.1 Purpose	7
1.1.1 Problem Statement	7
1.1.2 Problem Context and Motivation	8
1.1.3 Overview of Automation Need	9
1.2 Summary of Solution	9
1.2.1 System Overview	9
1.2.2 Integration of Sensors, Camera, and Database	9
1.3 Key Features	10
1.3.1 License Plate Recognition (LPR)	10
1.3.2 Cargo Classification	10
1.3.3 Weight Stabilization Detection	10
1.3.4 Centralized PostgreSQL Database	11
1.4 Impact	11
1.4.1 Operational Efficiency	11
1.4.2 Safety Improvements	11
1.4.3 Scalability and Future Applications	12
1.5 Conclusion	12
2. Introduction	13
2.1 Project Identification and Significance	13
2.1.1 Background and Industry Context	13
2.1.2 Team Motivation and Learning Objectives	13
Dawson Holmes	13
Anant Kumar	15
Noah Albert	15
Chinmayi Achanta	16

Joseph Benno	17
2.1.3 Sponsor and Stakeholder Roles	18
2.2 Broader Societal Impacts	18
2.2.1 Environmental and Waste-Management Benefits	18
2.2.2 Worker Safety and Reduced On-Site Risk	19
2.3 Legal, Ethical, and Privacy Considerations	19
2.3.1 Data Security and Privacy for License Plates	20
2.3.2 Ethical Use of AI Models and Dataset Bias	20
2.3.3 Compliance with NTEP and Safety Regulations	20
3. Project Characterization	21
3.1 Objective and Goals	21
3.1.1 Core System Goals	21
3.1.2 Stretch Goals and Future Expansion	22
3.2 Specifications and Requirements	22
3.2.1 Functional Requirements	22
3.2.2 Non-Functional Requirements	24
3.2.3 Hardware Requirements (Jetson Nano, Camera, Scale Simulator)	25
3.2.4 Software Requirements (YOLO, OCR, Postgres)	25
3.3 Concept of Operations	26
3.3.1 System Workflow Overview	26
Figure 1: System Workflow Diagram	26
3.3.2 Weigh-In / Weigh-Out Process	27
3.3.3 Data Flow from Capture to Database	30
3.3.4 Operator Interaction and Automation Triggers	31
4. Design Documentation	32
4.1 Research and Technical Decision Process	32
4.1.1 License Plate Recognition (YOLO + OCR)	32

Figure 2: High-Level LPR Architecture	33
Figure 3: YOLOv8 Model Variant Comparison	35
Figure 4: OCR Engine Comparison Table	37
4.1.2 Cargo Classification Model Research	39
Figure 5: Evaluation of Datasets	40
Figure 6a: MobileNetV3 Linear Probing Evaluation Result	41
Figure 6b: ResNet Linear Probing Evaluation Result	41
Figure 6c: EfficientNet Linear Probing Evaluation Result	42
4.1.3 Weight Stabilization Algorithm	43
4.1.4 Hardware-Software Integration Research	44
4.1.5 Database Design / Technology Research	45
Figure 7: Database Model Comparison Table	48
Figure 8: Database System Comparison Table	53
4.1.6 Backend Technology Research	53
Figure 9: Backend Technology Comparison Table	58
4.1.7 Database Design	59
4.1.8 API Design	60
4.2 System Diagrams	62
4.2.1 High-Level System Block Diagram	62
Figure 10: System Block Diagram	62
4.2.2 Use-Case Diagram	63
4.2.3 Activity and Sequence Diagrams	64
Figure 12: Weigh-In/Weigh-Out Sequence Diagram	64
4.2.4 Database Entity Relationship Diagram (ERD)	65
Figure 13: Transactions Entity Relationship Diagram	65
4.3 Subsystem Designs	66
4.3.1 Machine Learning & Computer Vision	66

4.3.1.1 YOLO Object Detection Pipeline	66
4.3.1.2 OCR Text Extraction (EasyOCR)	67
4.3.1.3 Pre-Processing and Post-Processing Steps	68
4.3.1.4 Cargo Classification Model	69
4.3.2 Sensors and Scale System	71
4.3.2.1 Scale Simulator Description	71
4.3.2.2 Weight Stability Detection Algorithm	72
4.3.3 Backend Integration	73
4.3.3.2 API Communication and WebSocket Setup	73
4.3.3.3 Authentication and Security Controls	73
4.3.4 Hardware Deployment	74
4.3.4.1 Jetson Nano Configuration	74
4.3.4.2 Camera Interface and Capture Logic	74
4.3.4.3 System Initialization and Remote Updates	74
5. Build, Prototype, and Test Plan	76
5.1 Build Overview	76
5.1.1 Development Environment Setup	76
5.1.2 Hardware Platform Specifications	77
5.1.3 Software Stack and Dependencies	78
5.2 Test Strategy and Results	80
5.2.1 Unit Testing (Individual Subsystems)	80
5.2.2 Integration Testing (End-to-End Pipeline)	80
5.2.3 Simulation Testing with Synthetic Data	81
5.2.4 Performance Testing on Jetson Nano	81
5.3 Performance Metrics	82
5.3.1 YOLO Detection Accuracy and Inference Time	82
5.3.2 OCR Accuracy and Confidence Scoring	83

5.3.3 System Latency (Capture → Database Storage)	83
5.3.4 Stability and Reliability Measurements	84
6. Results and Conclusions	85
6.1. Characterization of Results	85
6.1.1 License Plate Recognition Outcomes	85
6.1.2 Cargo Classification Results	86
6.2. Summary of Project	88
7. Administration	92
7.1. Budget and Financing	92
Figure 14: Budget	92
7.2. Agile and Scrum Artifacts	93
7.2.1 Project Management Overview	93
7.2.2 Product Backlog	95
7.2.3 Sprint 1 - Research	97
7.2.4 Sprint 2 - Environment Setup	98
7.2.5 Sprint 3 - Semester Wrap-Up	99
7.2.6 Senior Design 2 Sprint Plan	100
7.2.7 Setbacks and Resolutions	101
8. Acknowledgments	103
9. Bibliography	104

1. Executive Summary

1.1 Purpose

1.1.1 Problem Statement

Weigh stations, recycling centers, and waste management facilities rely heavily on accurate vehicle identification and weight tracking to process transactions, enforce regulations, and maintain operational efficiency. In the current “as-is” workflow, most facilities still depend on manual processes: operators visually read and record license plate numbers, estimate or classify cargo types by eye, and enter data into local systems or handwritten logs. These steps must be repeated both when the vehicle enters and when it exits the station, placing a large cognitive burden on employees and introducing numerous points of human error. The individuals most affected by this problem are scale house operators, facility managers, and organizations that depend on precise weight data for billing or compliance reporting.

The core issue is that this manual workflow is slow, inconsistent, and vulnerable to mistakes. Operators must simultaneously monitor incoming traffic, read rapidly moving license plates (sometimes obstructed, dirty, or poorly illuminated), verify cargo types, and determine when weight readings are stable enough to record. During busy periods, vehicles may queue, increasing pressure on operators to work quickly and increasing the likelihood of misrecorded plate numbers or incorrect cargo descriptions. Misidentifying vehicles or capturing weights too early or too late directly compromises the accuracy of transaction logs. These errors propagate through the system, leading to incorrect billing, disputes with customers, and administrative overhead to correct or audit records.

The consequences extend beyond clerical inconvenience. A single incorrect plate entry can break the link between a vehicle’s weigh-in and weigh-out, preventing a net weight calculation and forcing staff to intervene manually. Failure to detect the correct cargo type can violate internal policies or environmental reporting requirements. If operators capture unstable scale readings, the resulting weight discrepancies can cause trucks to be overcharged or undercharged and costing companies money,

damaging client relationships, and potentially violating industry standards such as those governed by NTEP (National Type Evaluation Program). Over time, the cumulative impact of these inaccuracies can result in thousands of dollars lost through billing adjustments, hours of manual re-entry, and operational delays.

Additionally, the manual process creates safety concerns. To confirm cargo types or read unclear plates, operators may need to step outside or approach moving vehicles. Facilities with high traffic volume face increased risk of collision or injury whenever staff must get close to trucks. Inconsistent procedures also make it difficult to onboard new employees, as each operator develops informal work habits that may deviate from recommended safety or data-handling practices.

In summary, the current weigh-station workflow suffers from three interconnected problems: unreliable manual data capture, operational inefficiencies that slow processing, and safety concerns stemming from the need for close physical inspection. These issues impact both daily operations and long-term accuracy of facility records. A solution must directly address these root causes by improving reliability, reducing human dependency, and ensuring consistent, high-quality data capture without increasing operator workload.

1.1.2 Problem Context and Motivation

Landfill weigh stations currently rely on manual processes to document vehicle weights, identify incoming and outgoing trucks, and verify cargo contents. This creates several issues like slow processing times, inconsistent recordkeeping, and safety risks for operators working late hours in isolated environments. Our motivation is to modernize this workflow by building an automated system that uses computer vision and embedded hardware to collect and store this information reliably without requiring constant human supervision. This project gives us the opportunity to apply what we've learned in machine learning, computer vision, embedded systems, and backend development to solve a real-world problem with significant operational benefits.

1.1.3 Overview of Automation Need

Automating the weigh station process helps eliminate human error, reduces the time operators spend interacting with each vehicle, and improves safety by allowing remote supervision rather than requiring personnel to remain on-site. As landfill traffic continues to grow, automation ensures that the system can scale efficiently, handle high throughput, and maintain consistent data quality. By incorporating license plate recognition, cargo classification, and automated weight stabilization detection into one workflow, the system provides a complete digital record of each transaction without manual data entry.

1.2 Summary of Solution

1.2.1 System Overview

Our solution is a fully integrated, embedded system that automatically detects when a truck's weight stabilizes, captures an image of the vehicle and its cargo, identifies the license plate number, classifies the cargo category, and stores the results in a centralized PostgreSQL database. The system is built around the NVIDIA Jetson Nano, which runs all computer vision models directly on-device. A USB-connected camera captures imagery, the weight simulator provides real-time readings, and the Jetson processes everything before transmitting structured data to the database. This pipeline removes the need for manual inputs and creates a reliable, timestamped transaction for each vehicle.

1.2.2 Integration of Sensors, Camera, and Database

The system works by combining data from three major sources: the scale, the camera, and the database. The scale provides continuous weight readings, which we analyze to determine when the truck is stationary. At that moment, the camera captures a frame that contains both the vehicle and its cargo. The Jetson Nano runs machine learning models to extract the license plate text and classify the cargo type. Once processed, all information (including date, time, weight, image paths, and detection results) is packaged into a structured transaction and sent to the PostgreSQL database.

through a secure connection. This creates a complete, searchable record for landfill operations.

1.3 Key Features

1.3.1 License Plate Recognition (LPR)

The license plate recognition system combines object detection and optical character recognition (OCR) to automatically read plate numbers from truck images. We use a YOLO-based model to locate the license plate within each frame and then pass the cropped plate region through an OCR engine. This two-step approach gives us flexibility (YOLO handles localization, while OCR handles text extraction). The goal is to ensure that the system can reliably read plates even when conditions aren't perfect, such as at night or when the angle and distance vary. LPR is a critical part of the pipeline because it uniquely identifies each vehicle in the transaction.

1.3.2 Cargo Classification

The goal of the cargo-classification system is to automatically identify the type of load carried by trucks coming in and out of the facility. Right now, the focus is on recognizing basic categories such as general waste, construction debris, paper, and wood. To make this work in real time, the model needs to be lightweight so it can run quickly on an edge device like the Jetson Nano. The model should also handle low-resolution images, since some configurations or situations may not have reliably high quality footage available. A small and efficient model helps ensure the system stays responsive and reliable during daily operations.

1.3.3 Weight Stabilization Detection

Weight stabilization detection is responsible for determining when a truck has fully entered the scale and come to a complete stop. Instead of capturing images continuously, the system monitors incoming weight readings from the scale simulator and evaluates their stability over a short time window. If the readings remain within a small variance range for a predetermined duration (usually 2 - 3 seconds), the system considers the weight "stable". This event triggers image capture and launches the

processing pipeline. This feature ensures accuracy, prevents premature captures, and aligns the visual data with the correct weight measurements.

1.3.4 Centralized PostgreSQL Database

Regarding the database for this project, we are utilizing PostgreSQL, also known as Postgres. Postgres is a popular object-relational database system utilized for a variety of reasons. Primarily used in many dynamic web applications, as well as data analytics and social media platforms, many companies and developers use Postgres for its high reliability and robustness. This is our main reason as well. Strength and reliability are key for our data organization, especially regarding a consistent stream of customers (vehicles) entering the database, as well as simple and reliable transaction acquisition from said database.

1.4 Impact

1.4.1 Operational Efficiency

Automating the weigh station process significantly reduces bottlenecks in daily landfill operations. Since all major steps (weight reading, license plate capture, cargo classification, and data entry) are handled by the system, operators no longer need to manually log information for each vehicle. This lowers the chances of transcription errors and eliminates delays caused by slow or inconsistent data entry. The automated workflow ensures each truck is processed in a predictable and optimized way, leading to better throughput and smoother traffic flow at the landfill. In the long term, the centralized dataset also supports trend analysis, capacity planning, and reporting, which further enhances operational decision-making.

1.4.2 Safety Improvements

Safety is one of the biggest motivators for this project. Many landfill weigh stations operate late at night or early in the morning, when staffing levels are low and surrounding environments may be unsafe. By automating the data collection process, we reduce the amount of time operators must physically be at the station, especially during high-risk hours. This lowers the exposure to potential accidents, hostile

encounters, or environmental hazards such as poor weather and low visibility. The system also ensures consistent weight and image capture without requiring workers to walk near moving trucks, creating a safer monitoring environment overall.

1.4.3 Scalability and Future Applications

One of the strengths of our design is its modular and scalable architecture. Since each component runs on embedded hardware and communicates with a centralized database, the system can be deployed across multiple weigh stations with minimal adjustments. As landfill facilities grow or add new service lines, the same core system can be extended to support additional sensors, upgraded machine learning models, or new database features. Beyond landfills, this system can be adapted to other industries that rely on vehicle identification and load tracking such as recycling centers, construction drop-off sites, or agricultural weighing facilities.

1.5 Conclusion

The automated landfill weigh station system addresses real operational challenges through an embedded, intelligent, and reliable approach. By integrating machine learning, sensor data, and a robust database, the system creates a complete end-to-end record of every vehicle while improving safety and efficiency across daily landfill operations.

Our final product will be a fully functional prototype running on the NVIDIA Jetson Nano that can detect when a truck's weight stabilizes, capture images of the vehicle and its cargo, automatically read the license plate, classify the type of load, and securely store all relevant information in a centralized PostgreSQL database. The system will operate autonomously with minimal human interaction and demonstrate the full workflow from weight detection to data logging.

This solution forms a strong foundation for future enhancements and opens the door to greater automation within waste-management infrastructure.

2. Introduction

2.1 Project Identification and Significance

2.1.1 Background and Industry Context

Landfill weigh stations are essential for tracking vehicle loads, recording disposal weights, and maintaining accurate operational and regulatory records. Many facilities process a steady flow of commercial, residential, and industrial trucks each day, yet most stations still rely heavily on manual procedures. Operators typically identify vehicles by sight, record license plates by hand, verify cargo visually, and manually log weight readings.

These processes are slow, inconsistent, and vulnerable to human error, which can impact billing accuracy and compliance reporting. Safety is also a major concern as operators often work late at night in remote or poorly lit areas, increasing the risk of accidents or unsafe encounters.

At the same time, the waste-management industry is pushing toward modernization and automation. Facilities are seeking solutions that reduce labor demands, improve data integrity, and allow for more efficient movement of vehicles through the station. Technologies like computer vision and embedded systems create a clear path for this shift by enabling accurate, automatic data capture without constant human oversight.

Our project aligns directly with this industry need. By automating license plate recognition, cargo identification, and weight stabilization detection, we are contributing to a broader effort to make landfill operations safer, more efficient, and more data-driven.

2.1.2 Team Motivation and Learning Objectives

Dawson Holmes

My main motivations for choosing this project lie in its potential. The use of an automated vehicle weigh-in / weigh-out system has plenty of benefits compared to a

manual system, such as efficiency, cost-effectiveness, and general safety. This also can create more potential for growth in this area. One of my family members owns a trucking company, so I can directly see the people that would benefit from a technology like this one. Another motivation for this choice is the type of experience I can gain from a project like this. I have an interest in backend and database management, and this seems to be the type of project that would meld great with that interest, in a fun and valuable way.

Before this project I primarily had experience with frontend development. This meant that I did a considerable amount of work with frameworks like HTML Bootstrap, as well as CSS styling and JavaScript functionalities. This project, as per our sponsors, requires no frontend up to this point, so I figured this would be a great segway into learning how to properly create and manage the backend of such an extensive project. This also included tacking on database implementation to my workload, which I was more than happy to do. This decision provided me with a deeper understanding of specific backend to database pipelines; giving me access to the entire backend and database of this project, which is helping my understanding of how every aspect comes together in the background.

The process of creating the database and implementing the backend was relatively simple to learn, but still hard to execute as I had never done this before. This is one of the main reasons as to why I wanted this role; to give me a fuller understanding of web development, building my skills in the backend half that I lack in comparison to frontend. This is proving to be the case. Just in this semester alone working on the backend and database for this project has taught me such an extensive amount, more than I initially predicted to get out of this project, and has made me reconsider a lot of my current motivations in regards to software development, giving me a more well-rounded skillset and taking me down different roads than I previously aspired to go down.

Anant Kumar

I chose this project because it aligns perfectly with my interest in embedded systems and edge computing architecture. While cloud-based software has its place, I am far more fascinated by the challenge of building fully self-contained solutions that can solve or speed up real world problems. The concept of "edge AI", where the processing power and decision-making live directly on the device rather than a remote server, also interests me. The Vehicle Weigh System allows me to design and build an autonomous unit that functions fully in the field, allowing me to explore my interests and learn more about them at the same time.

This project also aligns with my chosen career path as well. Despite computer science being largely rooted in web development or software design. Over time, I realized that I found the aspect of abstracted systems and browser based development to be boring. I wanted to move closer to the hardware and interact with it directly. I specifically chose the Hardware-Software Communication role to bridge that gap. It forces me to tackle low-level challenges, such as using raw sockets and legacy protocols to parse continuous data streams from industrial scales, while also dealing with the abstraction aspect, routing data to the correct parts of the system such as the AI/ML part.

Working with the Jetson Orin Nano also allows me to engage directly with the infrastructure required for Machine Learning. I am deeply interested in AI and edge computing applications of AI. By building a pipeline that feeds physical camera data into our computer vision models, I get to see exactly how edge devices can allow AI to make tedious tasks such as weigh stations more automated. This project gives me the chance to move away from the cosmetic side of software development and learn more about the full-stack development of edge devices.

Noah Albert

The Vehicle Weigh-in Weigh-out system was a fascinating project for me, so I selected it as my top choice in the team selection process at the beginning of the Senior

Design 1 course. I have been interested in the environment and all the moving parts of society that work to allow us as humans to try to live harmoniously with the world around us, or at least work towards minimizing the negative impacts that our consumerist and industrial society bring about just by their nature. One of the most direct ways that an impact is made in this area is through waste management, collection and storage. To be able to work in this sector has a great draw for me as it is the forefront of environmental protection, and I am elated to be able to make an impact.

Additionally, I have always been interested in working on embedded systems, fascinated by the principles of designing an all-in-one hardware based solution. I was motivated to work on this project as it is an opportunity to work with building around specific hardware, namely the Jetson Orin Nano, and create a unique and portable solution.

Recent trends in computing have had a large focus on automation. This project is centered around automating repetitive tasks done by an operator. As someone who has worked menial data-entry jobs before, the idea that someone or hopefully many people will be liberated from such tasks is very compelling to me.

I hope to become more familiar with websocket integrations, working with mini-computers such as the Jetson Orin Nano, and processing raw data. I also hope to learn generally about developing and deploying a discrete system solution such as the device we are developing. In addition to development and deployment, I hope to learn about maintaining solutions like this and specifically about firmware over the air.

Chinmayi Achanta

This project motivates me because it gives me the chance to take the skills I've been learning (especially in machine learning and computer vision) and put them into a real system instead of just another class assignment. I've always liked the idea of building something end-to-end and seeing it actually work in the real world, and this project finally gives me that opportunity. Starting from the ground up and watching the system grow into a functional product is something I've genuinely looked forward to.

I'm especially excited about the ML side of the project. Working on license plate recognition and cargo detection fits perfectly with what I enjoy and what I already have some experience with. At the same time, running these models reliably on embedded hardware like the Jetson Nano is something completely new to me, and I'm looking forward to that challenge. It's one thing to train a model on a laptop; it's another to integrate it into a full pipeline that needs to work in real time. That part really pushes me to grow my skills and learn how ML fits into a larger system.

I also like that this project is open-ended and student-driven. We have room to explore different approaches, make important design decisions, and really shape the final product ourselves. On top of that, the fact that this system can improve safety and efficiency for operators makes the work feel meaningful and motivates me to contribute my best. My goal is to use my ML experience to build components that strengthen the entire system and help bring the final product together.

Joseph Benno

I chose this project because of an AI course I took a few years ago that first got me interested in how machine learning can be used to solve real, everyday problems. When this project came up, it felt like the right chance to move beyond classroom examples and actually apply what I've learned to something that impacts real operations and real people. My main role in the team centers around machine learning and computer vision, and this project lines up well with the skills I want to grow.

I am also taking a computer vision course this semester, which gives me another environment to practice and test what I'm building for this system. Working on the cargo-classification pipeline, license-plate recognition, and the integrations around them lets me combine academic concepts with hands-on engineering. Overall, my goal is to walk out of this project with a stronger understanding of how to design, train, and deploy ML models in real hardware systems not just in theory, but in a way that actually works end-to-end.

2.1.3 Sponsor and Stakeholder Roles

Our project involves several key sponsors who guide and support the development process from both technical and organizational perspectives.

Scott Piersall, our industry sponsor, provides the core project requirements and operational expectations. He helps us understand how landfill weigh stations function, what challenges operators face, and what the final system needs to accomplish. Scott also supplies the simulation tools and real-world context that keep our design aligned with industry needs.

Dr. Liqiang Wang, our faculty advisor, oversees the administrative side of the project. He ensures we stay on track with Senior Design requirements, helps coordinate communication with our sponsor, and provides general guidance on documentation and project management.

Dr. Zihang Zou, our technical advisor, offers direct hands-on support. He provides access to the Jetson Nano and camera. Dr. Zou also assists with technical planning, hardware integration, and performance considerations, helping us turn our research into a realistic embedded system.

Together, Scott, Dr. Wang, and Dr. Zou ensure that our project remains technically feasible, academically sound, and relevant to real-world industry workflows.

2.2 Broader Societal Impacts

2.2.1 Environmental and Waste-Management Benefits

The intended goal of our device is to streamline the vehicle weighing process that occurs at many waste management sites across the country. Currently much of this process is done manually, and through automation we can hope to not only improve speed and throughput but also improve accuracy through reducing human errors.

There are many laws and regulations put in place regulating the volume and rates at which these waste management plants can accept waste. These laws have been designed with the protection of the environment and the sustainability of the waste

management solution in mind, therefore improving the accuracy at which the waste management plants comply with these regulations will be beneficial to both the environment and the waste management plant.

2.2.2 Worker Safety and Reduced On-Site Risk

Automating the weigh station process directly improves worker safety by reducing the need for operators to stand near large vehicles and active traffic lanes. Traditional weigh stations require staff to verify trucks, record information, and monitor the scale in person, often during late-night hours or in poor weather conditions. These situations increase the risk of accidents and unsafe interactions in isolated areas.

By allowing the system to handle identification, data capture, and logging automatically, operators can supervise the process remotely instead of remaining on-site at all times. This minimizes exposure to hazardous conditions and reduces the chance of vehicle-related incidents or security concerns during low-visibility hours. Overall, automation creates a safer working environment by shifting the highest-risk tasks from people to technology.

2.3 Legal, Ethical, and Privacy Considerations

Ethical concerns on this project are limited, however not completely non-existent. We do not want to completely remove operator control as it is unreasonable to expect that there will never be errors or erroneous measurements. A possible compromise for this might be requiring special permissions for modifications or for administrator/supervisor approval for any modifications.

When considering privacy, the system is mostly concerned with waste that does not have any sentience or concern for privacy. However, we will be pointing one or more cameras at each truck which comes through, and depending on the camera's perspective we might incidentally capture footage of the truck driver as they are driving through the weigh station. We will only be identifying the footage by the truck and should not pay any attention and certainly not have any identification of the drivers, but in the case that their likeness be captured by the cameras, we will need to ensure that

these videos are stored safely in the database and deleted from local storage after upload, and that database access is restricted and protected.

2.3.1 Data Security and Privacy for License Plates

With all images captured, they will be temporarily stored on device for processing, then uploaded to our secure SQL database. The license plate information will only be used to identify which truck or vehicle is currently being weighed, and should not interact with any registries outside of our own.

2.3.2 Ethical Use of AI Models and Dataset Bias

As previously discussed, incidental images of drivers or other people will not be analyzed as our models will only be considering text read on the license plates. Ethical concerns that may apply to people do not necessarily extend to text read off of license plates. Our optical character recognition models will likely be biased from training on license plates and may struggle with identifying other text such as non-license plate identifiers on trucks.

2.3.3 Compliance with NTEP and Safety Regulations

Because our system interacts with certified weighing equipment, it must align with the requirements set by the National Type Evaluation Program (NTEP). While our prototype operates through a simulator rather than a physical certified scale, the design principles still need to reflect NTEP-compliant practices, especially around stable weight detection, consistent data capture, and avoidance of human interference during measurement.

In addition to weight regulations, the system must follow general safety standards for industrial environments. This includes ensuring that automated processes do not interfere with the scale's mechanical operation, maintaining clear separation between digital components and physical equipment, and securely handling any data collected from vehicles. By designing the system with these requirements in mind, we help ensure that future deployments can integrate smoothly with certified scales and operate safely in real-world landfill environments.

3. Project Characterization

3.1 Objective and Goals

3.1.1 Core System Goals

The primary goal of this project is to develop an automated weigh station system capable of reliably identifying vehicles, capturing cargo information, and recording accurate weight measurements without requiring continuous human involvement. To achieve this, the system must integrate multiple components (computer vision models, sensor data, embedded hardware, and a centralized database) into one cohesive, real-time workflow. Our core goals include:

- **Automated License Plate Recognition (LPR):** Accurately detect and read license plates from incoming and outgoing vehicles using a combination of object detection and OCR. This provides a unique identifier for each transaction and eliminates the need for manual documentation.
- **Cargo Classification:** Identify general types of cargo carried by trucks using lightweight, fast ML models suitable for the Jetson Nano. Even coarse-level classification supports recordkeeping, waste categorization, and potential auditing processes.
- **Weight Stabilization Detection:** Continuously monitor scale readings and determine when a truck has come to a complete stop. Once the weight stabilizes, the system triggers image capture and data processing to ensure each record aligns with an accurate measurement.
- **Integrated Data Logging:** Store all relevant information (license plate text, cargo classification, stabilized weight, timestamp, and captured images) in a centralized PostgreSQL database. This ensures reliable and searchable transaction histories.
- **Real-Time Operation on Embedded Hardware:** Run all core processing locally on the NVIDIA Jetson Nano to ensure the system can function independently without relying on cloud services or high-bandwidth connections.

3.1.2 Stretch Goals and Future Expansion

Looking ahead, one of the major stretch goals for this system is supporting deployments across multiple facilities, each with different hardware setups. In real landfill and recycling environments, no two stations are identical. Camera placement, viewing angle, lighting conditions, and even the type of industrial scale used can vary from site to site. A future version of the system could add simple setup tools that adjust to each site automatically, including camera angle calibration and basic environment checks, so the same software can run at different companies without needing a lot of manual tweaking.

Another big area for future improvement is making the machine-learning models more reliable in real conditions. The current system performs well on clear, front-facing images, but real-world stations often deal with lower-quality visuals, motion blur, tough lighting, and weather effects like rain or glare. Future versions of the model could use stronger and more varied training data, along with techniques that help it adapt to different environments or combine views from multiple cameras when needed. There is also room to test lighter, faster model architectures and use methods like active learning so the system keeps improving as it sees more real-world examples.

3.2 Specifications and Requirements

3.2.1 Functional Requirements

These requirements describe what the system must do in order to meet its operational goals. They outline the core behaviors the system must perform, such as capturing weight data, recognizing license plates, classifying cargo, and storing each transaction in the database. Together, these functionalities ensure the system can fully support an automated weigh-in/weigh-out workflow.

Weight Detection & Stabilization:

- The system shall continuously read weight values from the scale.
- The system shall detect when the weight has stabilized based on variance thresholds over a fixed time window.
- Once stabilized, the system shall trigger the capture and processing pipeline.

Image Capture & Processing:

- The system shall capture an image of the truck and cargo at the moment the weight stabilizes.
- The system shall isolate the region containing the vehicle's license plate.
- The system shall extract the license plate text using OCR.

Cargo Classification:

- The system shall process the captured image to identify the general cargo category (construction debris, paper, wood, mixed waste).
- The system shall use a lightweight ML model capable of running on the Jetson Nano.

Integrated Transaction Logging:

- The system shall combine image, license plate text, cargo type, date, timestamp, and weight into a single transaction record.
- The system shall insert each completed transaction into a PostgreSQL database.

Data Access & Security:

- The system shall connect securely to the central database using sponsor-provided credentials.
- The system shall store data only in authorized tables and ensure that each client can access only their own records.

Edge Device Operation:

- The Jetson Nano shall run the full pipeline (detection, OCR, classification, data logging) without requiring cloud execution.
- The system shall support remote firmware updates as provided by the sponsor.

3.2.2 Non-Functional Requirements

These requirements define how well the system must perform and under what constraints.

Performance Requirements:

- The system shall process each transaction within a reasonable timeframe (target: ≤ 2 seconds per event).
- ML models shall be optimized for embedded inference using techniques such as ONNX conversion, quantization, or TensorRT.

Accuracy Requirements:

- The license plate recognition system shall correctly identify plates under typical lighting and positioning conditions.
- The cargo classification model shall achieve acceptable accuracy on coarse-level labels suitable for landfill operations.
- Weight stabilization must reliably trigger only when the vehicle is fully stopped.

Reliability Requirements:

- The system shall operate continuously for extended periods without manual restarting.
- Database transactions must be atomic to prevent incomplete or inconsistent entries.
- The system shall handle intermittent network disruptions without data loss.

Security Requirements:

- All database access must be authenticated using sponsor-provided credentials.
- The system shall not store raw license plate data locally once transmission to the database is complete.
- Data transmission must follow best practices to prevent unauthorized access.

3.2.3 Hardware Requirements (Jetson Nano, Camera, Scale Simulator)

The system relies on three primary hardware components that enable real-time data capture, processing, and event triggering.

NVIDIA Jetson Nano:

- Serves as the embedded computing platform responsible for running the license plate recognition, cargo classification models, and weight stabilization logic.
- Must support GPU-accelerated inference, Python execution, and local storage for temporary image handling.
- Requires stable power and network connectivity to communicate with the PostgreSQL database.

USB Camera:

- Captures images of incoming vehicles and their cargo at the moment the weight stabilizes.
- Must provide consistent image quality under varying lighting conditions and support real-time video streaming to the Jetson.
- Requires a reliable mounting position that ensures clear visibility of license plates and cargo areas.

Scale Simulator:

- Provides continuous weight readings that the Jetson Nano processes to determine stabilization.
- Must output values at a sufficient frequency to detect motion or stability within a short time window.
- Interfaces with the Jetson through the communication protocol or API provided by the sponsor.

3.2.4 Software Requirements (YOLO, OCR, Postgres)

The software for this project needs to run entirely on-site without relying on any cloud services. Tools like YOLO, OCR, and PostgreSQL must stay lightweight so they can run on small edge computers such as a Jetson Nano or any similar machine

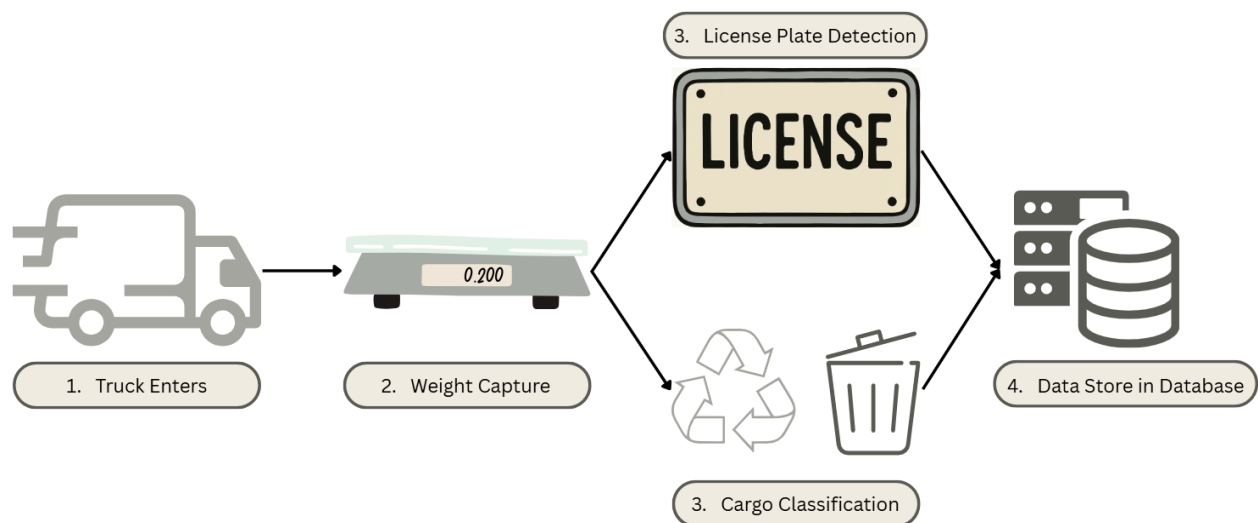
available at the facility. As long as the site has a working scale, a camera, and a local computer, the system should operate the same way. To make deployment easier and keep the setup consistent across different locations, the application will be packaged in containers so it can run the same on any supported device.

3.3 Concept of Operations

3.3.1 System Workflow Overview

The automated weigh-station system follows a straightforward end-to-end workflow designed to capture all necessary data from a truck as it enters, weighs in, and proceeds through the station. The process integrates weight readings, computer vision, and data logging into a single streamlined pipeline. Figure 1 (the diagram below) illustrates the major stages of this workflow.

Figure 1: System Workflow Diagram



1. **Truck Enters the Scale:** A truck drives onto the weigh station platform. As soon as the weight becomes non-zero, the system recognizes the presence of a vehicle. This triggers the weight monitoring loop, where the system continually reads data from the scale or scale simulator.
2. **Weight Capture and Stabilization:** While the vehicle settles on the scale, the weight naturally fluctuates. The system continuously analyzes these readings and waits for the weight to stabilize for a set duration (2 - 3 seconds). Once

stabilization is detected, the system initiates image capture and moves into the data-processing stage.

3. Vision Processing: License Plate Detection & Cargo Classification: After the weight is deemed stable, the Jetson Nano requests a camera frame and passes it to two computer-vision modules:
 - License Plate Recognition (LPR): Detects the plate region, then runs OCR to extract the plate number.
 - Cargo Classification: Analyzes the same frame to categorize the visible cargo into a general waste type (e.g., wood, paper, mixed debris).

Both outputs become part of the transaction record linked to this weigh event.

4. Data Storage in the Database: Once weight, license plate, cargo type, and timestamp are obtained, the system stores the complete transaction in the PostgreSQL database. For weigh-in events, this creates a new “pending” record. For weigh-out events, the system retrieves the previous weigh-in entry and computes the net cargo weight before saving the final result. The system then waits for the scale to return to zero before resetting back to the idle monitoring state, ensuring clean separation between vehicles.

3.3.2 Weigh-In / Weigh-Out Process

The vehicle weigh-in/weigh-out sequence is the core operational flow of our automated system. This process removes the need for manual data entry by automatically capturing, processing, and recording all relevant transaction information for each truck. The goal is to accurately determine the net cargo weight while also capturing license plate data, cargo type, timestamps, and weight information in a consistent and reliable way.

The complete cycle consists of several stages that work together to produce an automated and accurate transaction record:

1. Initial Arrival and Weight Detection (Weigh-In): When a truck arrives at the station, it drives onto the scale platform, which may be a dedicated inbound scale or a bidirectional one depending on the site. Once the truck’s weight registers,

the backend system (continuously polling the Weight Module) detects a non-zero value and transitions from idle monitoring into active processing. The system then begins reading incoming weight values in real time, watching for the natural fluctuations that occur while the truck settles.

2. **Weight Stabilization Monitoring:** During this phase, the backend queries the Weight Module at a frequent interval (approximately once per second). The system waits for the weight to remain unchanged for a configurable duration, such as three seconds. This ensures that the measurement is accurate and the truck has fully stopped. Capturing weight too early risks inaccurate or unstable values and could create compliance issues, so stabilization is an essential requirement of the process.
3. **Vision Processing and Data Capture:** Once a stable reading is detected, the system triggers two computer vision processes:
 - **License Plate Recognition (LPR):** The backend requests a high-resolution camera frame and passes it into the LPR model, which extracts the license plate region and utilizes OCR to convert the text into the vehicle's plate number. This number becomes the key identifier for linking weigh-in and weigh-out events.
 - **Cargo Classification:** The system also captures a frame for cargo analysis. The classification model assigns the load to one of several predefined categories (e.g., construction debris, paper, wood, mixed waste), reducing the need for manual inspection.

These outputs are collected alongside the stable weight to form the vehicle's weigh event.

4. **Transaction Type Determination and Database Operations:** The backend now checks the database to determine whether this stabilized event represents a weigh-in or weigh-out.

Scenario A - Weigh-In Transaction: If no prior pending transaction exists for this license plate, the system identifies this as a weigh-in. It creates a new record containing:

- License plate number
- Cargo type
- Gross (loaded) weight
- Timestamp
- Weight capture method (manual or automated)
- Direction (“In”)

This entry remains in a pending state until the vehicle returns for weigh-out.

Scenario B - Weigh-Out Transaction: If the database shows a previous open weigh-in record, the system treats this as the corresponding weigh-out. It retrieves the earlier gross weight and computes the net cargo weight by subtracting the current (empty-truck) weight. A new record is created that includes:

- License plate number
- Cargo type (carried over from weigh-in)
- Empty weight
- Weigh-out timestamp
- Direction (“Out”)
- Calculated net weight (via prior record lookup)

5. Scale Clearance and Cooldown: After storing the transaction, the system continues monitoring the scale until it returns to zero (or a near-zero threshold). This cooldown period prevents false triggers, ensures the previous truck has fully exited, and avoids overlapping or duplicated transactions.
6. Return to Idle State: Once the scale clears, the backend transitions back into its idle state, continuously polling for the next non-zero weight reading. At this point, the entire cycle resets, ready for the next truck to enter.

3.3.3 Data Flow from Capture to Database

Once the system detects that the scale has produced a stable reading, the data-handling pipeline begins. This section focuses specifically on how information moves through the system from the moment an image is captured to when the finalized transaction is stored in the PostgreSQL database.

1. **Triggering the Data Pipeline:** A stable weight reading acts as the signal for the system to collect all relevant data. This trigger ensures that every processing step is aligned with a valid, settled measurement.
2. **Vision Data Processing:** After triggering, the Jetson Nano requests a single camera frame and sends it through the two computer-vision modules:
 - **License Plate Recognition:** Extracts the plate region and converts it to text using OCR.
 - **Cargo Classification:** Predicts the cargo category from the same frame.Both modules return lightweight outputs (text for the license plate and a categorical label for the cargo type).
3. **Data Packaging:** The backend combines the stabilized weight, timestamp, and the two vision outputs into a structured data bundle. At this stage, no heavy computation is done beyond basic formatting. This keeps the pipeline efficient and suitable for real-time operation.
4. **Database Update:** The packaged transaction is sent to the PostgreSQL database using secure credentials provided by the sponsor.
 - If it is a new vehicle event, a weigh-in entry is created.
 - If it corresponds to an existing weigh-in, the system completes the weigh-out record and stores the final net weight.Only validated and complete data is inserted to maintain database consistency.
5. **Ready State Reset:** After the database confirms the write operation, the system clears temporary buffers and returns to its monitoring state, waiting for the next non-zero scale reading.

3.3.4 Operator Interaction and Automation Triggers

While the system operates autonomously for most transactions, operators can still maintain oversight and intervene when necessary. The system uses several triggers to automatically transition between states and execute operations.

Weight-Based Triggers:

- Truck Arrival: Non-zero weight above a set threshold exits the system from an idle loop and activates processing mode
- Stabilization: Weight steady 3 - 5 seconds triggers a weight capture and image and vision processing
- Scale Clearance: Weight returns below set threshold returns system to idle state

Confidence Threshold Triggers:

- Low confidence predictions flag transaction for manual review, allowing errors to be caught and detected

System Health Triggers:

- Automated health checks to verify scale connection, camera status, database connectivity, and system resources
- Failed checks can trigger notifications or halt automation to ensure smooth operation

Configuration and Calibration: The system can be configured during initial setup or when conditions change:

- Weight stability thresholds
- Confidence score minimums for manual review
- Camera positioning and exposure settings
- Scale connection information (IP address, port)

4. Design Documentation

4.1 Research and Technical Decision Process

4.1.1 License Plate Recognition (YOLO + OCR)

Introduction and Role in the System

The License Plate Recognition (LPR) module serves as one of the foundational components of our automated weigh station system. Its primary purpose is to identify each vehicle that enters and leaves the landfill scale by extracting its license plate number from the camera feed. This identification is critical because it enables the system to link weigh-in and weigh-out events, match individual trucks to their transaction histories, and compute accurate net cargo weights. At traditional weigh stations, this process is performed manually usually by reading plates on the fly or handwriting plate numbers into a logbook. This approach is slow, inconsistent, and vulnerable to human error. By automating plate recognition, our system not only speeds up the process but also improves data accuracy and reduces operator workload.

In our pipeline, LPR is triggered immediately after the system detects that the scale reading has stabilized. This timing is intentional: stable weight typically means the truck has come to a complete stop, reducing motion blur and ensuring that the captured frame is suitable for detection and OCR. Once activated, the LPR component processes a single camera frame, identifies the plate region, extracts the text, and returns a clean alphanumeric string. This string becomes the unique identifier used for transaction creation in the PostgreSQL database. Because every weigh-in and weigh-out depends on this identifier, the reliability of the LPR module directly affects the overall integrity of the system.

To ensure strong performance on the Jetson Nano and in variable outdoor conditions, we chose a modular LPR architecture built around two proven technologies: the YOLOv8 object detection model and the EasyOCR text recognition engine. These tools were selected after extensive research into modern plate-recognition systems and the constraints of an embedded deployment environment.

Two-Stage LPR Architecture (Detection + OCR)

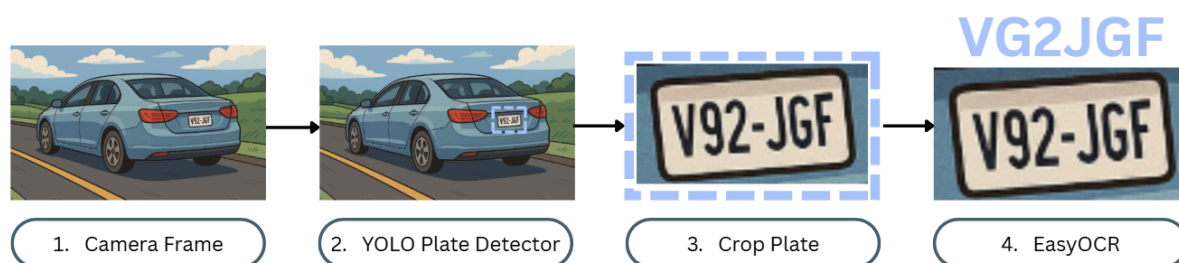
After surveying multiple academic sources, open-source projects, and commercial LPR systems, we found that nearly all modern solutions follow a similar two-stage pipeline shown in Figure 2:

- License Plate Detection - Locate the plate within the full vehicle image.
- Optical Character Recognition (OCR) - Extract the characters from the cropped plate region.

This architecture is widely used because it is robust, interpretable, and easier to optimize. Each stage can be developed and evaluated independently, which is extremely advantageous for debugging and model improvement. For example, if the OCR output is incorrect, we can determine whether the issue is caused by poor cropping (detection stage) or poor character recognition (OCR stage). This level of transparency is essential for a senior design project where iteration and improvement are expected.

Additionally, the two-stage structure gives us the flexibility to change or retrain individual components without affecting the entire system. For instance, we can swap the detector model or fine-tune the OCR engine without retraining the other part of the pipeline. This modularity aligns well with the long-term vision of the system, where future sponsors or operators might need to adapt the LPR model to new lighting conditions, camera types, or plate formats.

Figure 2: High-Level LPR Architecture



This diagram illustrates the two-stage structure of our LPR pipeline. YOLO first detects the license plate within the full camera frame, and the cropped region is then passed into EasyOCR for text extraction.

YOLO-Based License Plate Detection

The first stage of our LPR pipeline focuses on locating the license plate within the full image captured by the camera. Because outdoor environments introduce unpredictable variables such as varying camera angles, vehicle speeds, dirt, shadows, and lighting changes, the plate detector must be fast, reliable, and resilient. After reviewing several object detection families (including Faster R-CNN, SSD, RetinaNet, and YOLO) we selected YOLOv8 as the basis of our detection stage due to its strong balance of speed and accuracy, especially on embedded hardware.

YOLO (“You Only Look Once”) models are designed for real-time object detection and are known for processing the entire image in a single pass through the neural network, which significantly reduces inference time compared to other methods. This single-shot design makes YOLO particularly well-suited to embedded systems like the Jetson Nano, where efficiency and low latency are essential. In a weigh station, the system must be able to respond quickly once weight stabilizes (ideally within a couple of seconds) so the detector cannot afford to introduce delays.

Although YOLO has gone through many iterations over the years, YOLOv8 offers a number of architectural improvements such as anchor-free detection, decoupled heads, and enhanced feature extraction layers. These improvements help YOLOv8 perform well on small or thin objects, which is important because license plates typically occupy only a small portion of the overall frame. The anchor-free design also makes it more flexible when plates appear at unconventional angles or distances.

Within the YOLOv8 model family, we evaluated several size variants (shown in Figure 3) including YOLOv8n (nano), YOLOv8s (small), and YOLOv8m (medium). For our system, we selected YOLOv8n because it is optimized for edge devices and offers the fastest inference time. Although the model sacrifices some accuracy compared to larger variants, the trade-off is appropriate for our deployment environment. The Jetson Nano has limited GPU memory and compute power, so using smaller models helps keep inference fast enough to support near real-time processing.

Selecting YOLOv8n also gives us room to apply further optimizations such as TensorRT acceleration, FP16 quantization, or converting the detector to ONNX format, all of which can improve performance without drastically impacting accuracy. These optimizations will be especially important once we begin testing with real camera input from the Jetson Nano and start benchmarking inference times under practical conditions.

Figure 3: YOLOv8 Model Variant Comparison

Model Variant	Parameters	Speed	Accuracy	Jetson Suitability
YOLOv8n (XS)	~3.2M	★★★★★ ★	★★★★☆☆	Ideal
YOLOv8s (S)	~11.2M	★★★★★ ☆	★★★★☆	Possible with optimization
YOLOv8m (M)	~25.9M	★★★★☆ ☆	★★★★★	Too slow for real-time use
YOLOv8l (L)	~43.7M	★★☆☆☆ ☆	★★★★★ +	Not suitable for Jetson Nano
YOLOv8x (XL)	~68.2M	★★☆☆☆ ☆	★★★★★ ++	Not feasible

Comparison of different YOLOv8 model sizes. Due to hardware constraints on the Jetson Nano, YOLOv8n provides the best balance between inference speed and sufficient detection accuracy.

Optical Character Recognition (OCR) Engine Selection and Comparison

After the plate detection stage identifies and crops the license plate region, the next step in the LPR pipeline is to extract the alphanumeric characters printed on the plate. This requires a reliable Optical Character Recognition (OCR) engine capable of handling the wide range of conditions encountered at landfill weigh stations. OCR is

crucial because even perfect plate detection becomes useless if the extracted text is inaccurate or unreliable. Throughout our research, we evaluated multiple OCR tools, focusing on those commonly used in real-world LPR applications and those capable of running on edge hardware like the Jetson Nano.

Two major OCR engines emerged as leading candidates: Tesseract and EasyOCR. These engines follow very different philosophies: Tesseract is a traditional, rule-based OCR system, while EasyOCR is a deep-learning-based model trained on a broad variety of character patterns. Understanding their differences and their behavior under real-world conditions was essential for selecting the right tool for our system.

Tesseract has been widely used for decades and excels on clean, high-contrast text. It operates by applying image preprocessing (such as thresholding and binarization) and pattern matching to detect and interpret characters. However, Tesseract struggles with environmental noise, angled text, motion blur, and low-resolution images. These limitations make it less suitable for our deployment scenario, where many trucks may have dirty plates, non-standard fonts, or uneven lighting.

EasyOCR, on the other hand, uses a neural network architecture designed to recognize characters directly from raw pixel data. It performs significantly better on skewed, partially obstructed, or low-quality images because it learns visual patterns rather than relying solely on sharp text boundaries. EasyOCR also provides confidence scores for each prediction, giving us the ability to filter out unreliable outputs or request manual verification. While EasyOCR is more computationally intensive than Tesseract, the Jetson Nano's GPU allows us to run it efficiently, especially after model optimization.

Because of these factors, we selected EasyOCR as our primary OCR engine, with Tesseract kept as a lightweight fallback option. EasyOCR's deep learning approach aligns with the messy, unpredictable nature of real landfill environments, and its flexibility makes it easier to fine-tune once we receive real-world data from the sponsor.

Figure 4: OCR Engine Comparison Table

Feature	EasyOCR	Tesseract OCR
OCR Method	Deep-learning based	Traditional rule/feature-based
Performance on Noisy/Skewed Plates	Excellent	Weak
Supports Confidence Scores	Yes	Limited
Speed on Jetson Nano	Moderate (improvable with optimization)	Fast
Deployment Complexity	Medium	Low
Robustness to Lighting Issues	Strong	Weak
Suitability for Outdoor LPR	Highly suitable	Requires ideal images

Comparison of the two OCR engines considered for our system. EasyOCR outperforms Tesseract in nearly all real-world scenarios relevant to landfill and industrial environments.

Proposed End-to-End LPR Pipeline Design

The final step of our research and design process was to assemble all of the components (YOLO-based detection, preprocessing layers, OCR recognition, and validation) into a complete end-to-end pipeline that can operate efficiently on the Jetson Nano. Our goal was to create a modular architecture that supports real-time operation while remaining flexible enough to adapt to new data, hardware changes, or unforeseen environmental challenges encountered during deployment.

At a high level, the pipeline is activated only when the scale reports a stable weight. This ensures that camera captures occur under conditions where the vehicle has come to a stop, reducing motion blur and improving image consistency. Once activated, the Jetson Nano captures a full-resolution frame from the USB camera and sends it through the detection and recognition stack. Each stage of the pipeline performs a single, well-defined task, which makes testing and debugging straightforward. The pipeline is designed to operate in the following sequence:

1. **Frame Capture:** When the backend detects stable weight, it instructs the Jetson to capture a single image. This reduces computational overhead compared to processing continuous video.
2. **License Plate Detection (YOLOv8n):** The captured image is passed through YOLOv8n, which identifies the location of the license plate and returns a bounding box. YOLO's low latency ensures that this step completes quickly, even on embedded hardware.
3. **Region Cropping & Preprocessing:** The bounding box region is extracted from the full frame. Several preprocessing operations are applied, including contrast normalization, resizing, sharpening, noise reduction, and angle correction. These adjustments significantly improve the quality of the input fed into the OCR engine.
4. **OCR Stage (EasyOCR):** The preprocessed crop is passed into EasyOCR, which predicts the characters printed on the plate. EasyOCR also outputs a confidence score for each prediction, allowing the system to identify uncertain or low-quality results.
5. **Post-Processing & Validation:** The OCR output is cleaned and validated. This step includes removing invalid characters, enforcing alphanumeric plate formats, and discarding predictions below a certain confidence threshold. If the prediction is considered unreliable, the system can flag the event for manual review instead of inserting incorrect data into the database.
6. **Database Insertion:** The final plate string, along with associated metadata (timestamp, weight, cargo type, and image path), is stored in the PostgreSQL database, completing the weigh-in or weigh-out event.

Each of these stages is designed to be independently testable. For example, we can measure the accuracy of YOLO detection using known bounding boxes, evaluate OCR accuracy using labeled crops, or test preprocessing effects by measuring OCR confidence with and without normalization steps. This modular architecture makes the pipeline more maintainable and easier to refine as we move into the implementation phase.

Conclusion

Our research and design work for the License Plate Recognition module have provided a strong foundation for implementing a reliable, real-time LPR system tailored to the operational needs of automated weigh stations. By combining YOLOv8n for plate detection with EasyOCR for text extraction, and enhancing both with targeted preprocessing techniques, we have created a modular, flexible, and scalable architecture that can adapt to the unpredictable environmental conditions typical of landfill operations. This design aligns with the computational constraints of the Jetson Nano while remaining open to future improvements such as fine-tuning on sponsor-provided datasets or integrating alternative OCR engines.

4.1.2 Cargo Classification Model Research

Five lightweight image-classification models were tested for the cargo classification task: ResNet-18, MobileNetV3-Small, MobileNetV3-Large, EfficientNet-Lite0, and EfficientNet-B0. These models were selected because they are widely used on edge hardware and have relatively low parameter counts. ResNet-18, which has 11.7 million parameters, was included only as a baseline and removed early in the process because it performed the worst and ran the slowest across experiments. As shown in Figure 5, the evaluation used multiple public datasets to give a fair comparison across different conditions.

Figure 5: Evaluation of Datasets

Dataset	Size	# Classes	Class Labels
RealWaste	4,752	9	Cardboard, Food Organics, Glass, Metal, Miscellaneous Trash, Paper, Plastic, Textile Trash, Vegetation
Garbage Classification V2	19,762	10	Metal, Glass, Biological, Paper, Battery, Trash, Cardboard, Shoes, Clothes, Plastic
TrashNet	2,527	6	Glass, Paper, Cardboard, Plastic, Metal, Trash

Three public datasets were used for testing: RealWaste, Garbage Classification V2, and TrashNet. While these datasets do not match the exact cargo categories in this project, they are similar enough in structure and difficulty to provide a reasonable indication of how well each model handles mixed materials. Together, they cover a wide range of image challenges, including cluttered backgrounds, low-quality images, and controlled lab-style photos.

Linear probing was used to compare the models by freezing each backbone and training only a single linear classifier. This method provides a quick way to measure the quality of the extracted features without running full fine-tuning. It also reduces the risk of overfitting since the datasets are fairly small. Even with this lightweight approach, linear-probe performance usually falls within ten to fifteen percent of full fine-tuning, making it a reliable indicator of model strength.

Before discussing model behavior, the linear probing results are shown in Figures 6a-6c, each corresponding to a different model family. As shown in Figure 6a, both MobileNetV3-Small and MobileNetV3-Large performed well considering their size. The Small version, at 2.54 million parameters, handled easier datasets reasonably well but struggled on more challenging datasets like RealWaste and TrashNet. MobileNetV3-Large provided a noticeable improvement, often reaching ten to fifteen percent higher accuracy while staying compact at 5.48 million parameters. On the

ten-class GarbageV2 dataset, it exceeded ninety percent Top-1 accuracy, showing its stronger feature separation.

Figure 6a: MobileNetV3 Linear Probing Evaluation Result

```
=====
MobileNetV3 Results:
=====
```

Model	Dataset	Params (M)	Top-1 (%)	Top-5 (%)	Num Classes
mobilenetv3_small_100	realwaste	2.54	65.09	98.32	9
mobilenetv3_small_100	garbage_v2	2.54	86.64	99.19	10
mobilenetv3_small_100	trashnet	2.54	70.95	94.86	6
mobilenetv3_large_100	realwaste	5.48	78.23	99.26	9
mobilenetv3_large_100	garbage_v2	5.48	91.30	99.65	10
mobilenetv3_large_100	trashnet	5.48	80.43	99.80	6

As shown in Figure 6b, ResNet-18 produced the weakest results of all models tested. Despite having nearly twelve million parameters, it reached only thirty-eight percent Top-1 accuracy on RealWaste and failed to match the performance of both MobileNet and EfficientNet models. Although its Top-5 accuracy remained high, indicating the features were somewhat in the correct range, they were not as sharply separated as those produced by more modern architectures.

Figure 6b: ResNet Linear Probing Evaluation Result

```
=====
ResNet Results:
=====
```

Model	Dataset	Params (M)	Top-1 (%)	Top-5 (%)	Num Classes
resnet18	realwaste	11.69	38.28	93.48	9
resnet18	garbage_v2	11.69	73.67	97.07	10
resnet18	trashnet	11.69	51.58	93.48	6

Finally, Figure 6c shows that EfficientNet-Lite0 and EfficientNet-B0 consistently produced the best results across every dataset. EfficientNet-Lite0 (4.65M parameters) already surpassed MobileNetV3-Small by a wide margin. EfficientNet-B0 (5.29M parameters) delivered the strongest performance overall, beating MobileNetV3-Large on GarbageV2 while using fewer parameters. On TrashNet, EfficientNet-B0 reached

over eighty percent Top-1 accuracy and even achieved one hundred percent Top-5 accuracy, showing very clean feature separation.

Figure 6c: EfficientNet Linear Probing Evaluation Result

```
=====
EfficientNet Results:
=====
```

Model	Dataset	Params (M)	Top-1 (%)	Top-5 (%)	Num Classes
efficientnet_lite0	realwaste	4.65	74.03	99.26	9
efficientnet_lite0	garbage_v2	4.65	90.61	99.72	10
efficientnet_lite0	trashnet	4.65	79.64	99.60	6
efficientnet_b0	realwaste	5.29	76.34	99.47	9
efficientnet_b0	garbage_v2	5.29	91.80	99.82	10
efficientnet_b0	trashnet	5.29	80.24	100.00	6

EfficientNet-B0 ends up being the best balance of size and accuracy for something like the Jetson Nano. MobileNetV3-Large is still a strong choice if you want something fast and lightweight without dropping too much accuracy. MobileNetV3-Small is mainly useful when every bit of memory matters. ResNet18, while still a classic model, simply does not keep up with the newer designs in this type of evaluation.

A major reason EfficientNet-B0 performs so well is how the architecture was created. EfficientNet was designed through a large-scale search that tested many configurations to find the most efficient use of parameters. Instead of scaling in only one direction, it uses compound scaling, which grows the network's depth, width, and image resolution at the same time in a balanced way. This gives the model stronger and more expressive features without adding unnecessary size. Even the smallest version, B0, benefits from this design philosophy and often performs like a larger model despite being compact enough for edge devices.

ResNet takes a much older and simpler approach. Its main idea is stacking many residual blocks so deeper networks can train properly. This was groundbreaking at the time and still works well for general computer vision tasks, but it is not optimized for environments with limited compute or tight latency requirements. ResNet models tend to rely on depth to gain accuracy, which means you often need larger variants to reach the performance levels that efficient modern networks reach with far fewer parameters. In

our evaluations, ResNet-18's features were noticeably weaker, showing that its older design does not extract information as effectively for this type of problem.

For a project that needs real-time performance on a device like the Jetson Nano, these differences matter a great deal. EfficientNet-B0 provides high accuracy while keeping inference fast and memory usage low, making it ideal for live cargo classification. MobileNetV3 models also fit well in this space, especially when speed or very small model sizes are the priority. ResNet-18, however, becomes less practical, as it uses more compute while offering significantly lower accuracy. The combination of performance, efficiency, and real-time reliability is what ultimately makes EfficientNet-B0 the strongest overall choice for this system.

4.1.3 Weight Stabilization Algorithm

The weight stabilization algorithm is responsible for deciding when the truck is fully on the scale and the weight has actually settled, instead of grabbing a random value while everything is still moving. The scale constantly sends a stream of numbers. When a truck begins to drive onto the platform, the readings start near zero, then climb as more of the truck moves onto the scale, then hover around a higher value while the truck is parked, and finally drop back down toward zero as the truck leaves. The goal of the algorithm is to find that stable middle part and capture one reliable number from it.

To do this, the system keeps a small sliding window of the most recent readings, for example the last ten or twenty values. As each new reading comes in, the oldest one is dropped and the newest one is added to the window. The algorithm then looks at how much the numbers inside this window are changing. If they are rising quickly, it assumes the truck is still driving onto the scale. If they are falling, it assumes the truck is leaving. When the weight has climbed above a small minimum level and the numbers inside the window are all very close to each other for a short amount of time, the algorithm treats this flat section as a stable period and records the average value in that window as the truck's weight. Once the readings start dropping again, the system stops looking for a new stable point until the scale returns to its idle, near zero state.

This method helps the system ignore noisy parts of the signal, such as vibration, slight movements of the truck, or quick spikes that happen while it is entering or exiting the platform. Even if the raw stream of numbers looks messy, the algorithm focuses only on the calm, flat region between the rising and falling phases.

4.1.4 Hardware-Software Integration Research

Our research for hardware-software integration focused on establishing reliable communication channels between the Jetson Orin Nano and external devices, as well as creating a secure development environment for the team.

Remote Access and Network Configuration: Since the entire system relies on a single Jetson Nano for edge computing, establishing a secure method for the team to deploy and test code remotely was a priority. We debated between three solutions:

- Port Forwarding/Tunnels: We investigated exposing the Nano using Cloudflare Tunnels or ngrok or directly exposing a port. However, we rejected this approach because of security concerns, especially when factoring in the other preexisting code that was on the device.
- Traditional VPNs (WireGuard/OpenVPN): We researched OpenVPN and WireGuard. However, these solutions would still require us to set up a central server or bastion exposed on the internet that could route traffic to the Nano.
- Tailscale: A preexisting VPN solution. Our research into its abilities showed that it was able to handle NAT traversal when set up behind a team member's network, hole punching using a custom protocol to set up peer-to-peer connections while still falling back to a DERP relay when needed. This would allow one person to set up the Nano and everyone on our team to reliably and easily access it when necessary.

Scale Communication Protocols: Our research into scale integration evolved based on hardware availability and feedback from our sponsor:

- WebSocket Research: Initially, we investigated the WebSocket protocol, examining Python libraries such as websockets and Flask-SocketIO to handle real-time bidirectional data streams. We conducted experiments simulating

sending and receiving data to and from a websocket server/client, simulating the Jetson Nano connecting to the weight scale.

- Telnet Protocol: Following a sponsor meeting, we discovered the weigh scales actually use the Telnet protocol rather than WebSockets. Consequently, our research pivoted to RFC 854 (Telnet) and Python's telnetlib module. Here we used our existing knowledge from WebSocket research to figure out how to handle discrete data streams, managing the time between updates with a cache layer in-between the backend and weight scale, and parsing unstructured data with many different configurations into standardized weight readings.

Camera Interface: We also researched methods to interface with the camera to ensure low-latency video capture for the LPR and cargo classification pipelines:

- Hardware Interface: We confirmed the camera utilizes a USB connection rather than a CSI or network-based interface.
- Capture Libraries: We evaluated OpenCV as the primary capture solution. With research indicating that GStreamer is particularly promising for the Jetson platform due to NVIDIA's support for hardware-accelerated GStreamer plugins, which can significantly reduce CPU load during capture.

System Access and Storage Management: During the setup phase, we encountered access issues requiring a hard reset of the Jetson Nano's login credentials. This also necessitated research into the device's storage architecture and a basic understanding of Linux password management:

- Storage Media: We investigated the difference between EMMC and SD card storage handling. We initially attempted EMMC recovery methods but determined the device used an external SD card, allowing us to mount the Linux file system directly on a separate machine to modify the password shadow files.

4.1.5 Database Design / Technology Research

This project requires a simple, but effective database operation in order to store the relevant information we need regarding vehicle weighing. The nature of its design will consist of it being a transactional database, one that will store the transactions of

different customers (vehicles) coming in and being weighed. The different parts of the weighing process we need to store include the following: photo of the truck, timestamp, stable weight, license plate, cargo classification, weight capture method, and scale direction. Traditionally, this uses a relational database, as we can create more abstract relations between transactions and customers using ID numbers, but another storage system to be considered is the hierarchical database, due to its historical significance and current relevance in many major organizations.

We are required by our sponsors to use PostgreSQL, an object-relational database system. Because of this, we will primarily consider using a relational database as our main option. It is still useful to consider other roads of development, however; such as other database organizational models or management systems. This works because we need to get a sense of why we are using a specific technology, or to discover more effective ways to implement a specific database model.

Relational vs Hierarchical

The hierarchical structure is one of the more prominent database structures out there. Developed by IBM in 1968, it's one of the earliest models of data-storage and is still a relevant path to take in many current organizations. The nature of this type of database model looks like a tree structure, consisting of many parent-child relationships. It contains nodes that are connected by branches, with the topmost node being the root node. Each node has exactly one parent, and each parent can have multiple children.

This structure is very simple and intuitive. Its one-to-many relationship between different data elements makes it easier to understand and navigate, making it best suited for straightforward static frameworks where relationships between data are obvious. It can also provide extremely efficient record access given its hierarchical structure. This can give us predictable performance for queries, especially true for frequently accessed paths. This leads to one of its disadvantages: complexity. When a hierarchical database becomes larger and more complex, it becomes increasingly difficult to manage, leading to longer data requisition. When searching for another less frequent element, its top-to-bottom traversal can become time-consuming. This data

structure also cannot represent many-to-many relationships without duplicating the data. Another problem is that the nature of the one-to-many relationship structure prevents handling cases where one child has multiple parents. This prevents us from easily implementing the type of relationship structure we need, involving complex object relationships with multiple different attributes.

The relational structure is another prominent data structure to be considered for this database. First developed by E.F. Codd in 1970, this is the main choice for more modern applications, allowing for more complex relationships between many data elements at once. This type of model stores data in a tabular format, which consists of tables with rows and columns. A main difference with this model compared to the hierarchical one is that there are no physical links. It only deals with the data, not a physical structure, with each record having a unique ID leading to tables of related data.

This structure has advantages related to its complexity. It can have more complex relationships compared to a hierarchical structure, with its many-to-many relational structure. However, this leads to one of its major disadvantages. Relational databases can become very complex and time-consuming to design. Though these relationships can be a lot more robust, with not as much impact on record acquisition efficiency. One factor that allows this complexity is its flexibility. This flexibility gives us easier manipulation of its tables, allowing modifications without an entire restructuring of the database. These structural changes don't affect accessibility either, allowing the use of SQL to easily query data within a given database management system for accessibility. It is also much simpler to deploy, given its removal from physical storage. Along with simplicity, it has many tools and frameworks created by users skilled in database management, due to the nature of its popularity and use in more modern systems. Another advantage of the relational database model is its very nature adheres to ACID compliance. ACID stands for Atomicity, Consistency, Isolation, and Durability, and is a set of principles that any transactions within a database are reliably processed. This is important for the type of transactional database we are designing; It ensures data integrity maintains a view of reliability and trust.

Overall, the relational database is the ideal one to be considered, due to the nature of the data being stored. Compared with a more hierarchical database system, this object-relational database system works best with what we are trying to achieve. As you can see in Figure 7 below, we can easily see the comparisons between the two organizational models, showing the benefits of both. We want to have more of an abstraction between the actual data and the transaction itself, which is where the object-relational aspect comes in, using transaction ID's and customer ID's to relate each object (transaction) to the customer, as well as relate the relevant data being stored to the correct transaction and customer. This is also in line with the requirements of our sponsor as well.

Figure 7: Database Model Comparison Table

Model	Relational	Hierarchical
Data Structure	Table Related	Table Related
Scaling	Vertical and Horizontal	Only Vertical
Data Retrieval	SQL Queries	Top-to-Bottom Traversal
Performance	High / Lowers with Complex Joins	High with Common Paths / Lowers with More Depth
ACID Compliant	Yes	No
Use-Cases	E-Commerce / Data Analytics	File Systems / Charts

Given the structural nature of the relational model that we chose, this leads to the need of finding a relevant database software that can: handle the complexity of the data management, lend us effective management tools for complexity issues, and provide simple and efficient manipulation of created tables. Luckily, as stated previously, our sponsor requires the utilization of PostgreSQL, so we will be considering some other popular database software systems such as MySQL and Oracle in terms of comparison,

as well as potentially finding a database system that is more relevant to the project, and one that is easily learnable and relatively simple to use.

Database Systems: PostgreSQL

PostgreSQL is an open-source, object-relational database system first developed in 1986 as part of the POSTGRES project led by Professor Michael Stonebraker, developed at the University of California at Berkeley as a successor to the Ingres database. It was developed initially to provide better support for complex objects and better user extendibility for data types, operators, and access methods. SQL language capability was added in 1994 by Andrew Yu and Jolly Chen, and was subsequently changed to PostgreSQL and released as open-source. This was accomplished by adding an SQL interpreter to POSTGRES to essentially replace the initial implemented query language PostQUEL. Adding a SQL interpreter lined up with the previous and current standard for relational database management systems, and improved functionality within the system, allowing the database to scale and store complex data workloads more effectively.

PostgreSQL is one of if not the most popular database options out there for most applications. With the nature of it being open-source, it has an extremely robust support system underneath it, as well as being a free database technology. This database system has a strong reputation regarding its reliability, performance, and feature robustness. It stays within complete ACID compliance as well as staying as close to SQL standards as possible, leading to its reliability with enabling complex queries, subqueries, and joins. It can handle fairly large databases, with highly complex relationships between objects without any significant detriment to its performance. This allows it to be easily scalable with little performance drawbacks. Along with SQL capabilities, PostgreSQL also provides support for JSON and JSONB data types, allowing relevant developers to perform queries on data that is unstructured. It also provides NoSQL support, leading to its reputation of feature robustness.

With its feature robustness, however, leads to more complex system setup and maintenance. If you are not experienced with database systems, this has a bit of a learning curve compared to other systems out there. Another disadvantage is that it has

limited horizontal scaling without the help of extra tools, which can lead to issues in environments with heavy traffic. However, the previous advantages outweigh the disadvantages of a complicated setup and limited horizontal scalability, with a collection of incredible features and open-source framework.

Database Systems: MySQL

MySQL is another widely popular open-source object-relational database system first developed in 1995 by a Swedish company MySQL AB, founded by David Axmark, Allen Larsson, and Michael Widenius. The initial purpose of MySQL was for personal use from the system mSQL, being based on the language ISAM, a low-level language the creators at the time believed to have insufficient flexibility and too slow of a system. In 2008 Sun Microsystems bought MySQL AB, then subsequently Sun Microsystems was bought by the Oracle Corporation, making them the current developers of MySQL. After the Oracle acquisition, Michael Widenius branched the database system and created MariaDB.

One reason that leads to its popularity is that MySQL is free and open-source. It's not just open-source either; it has a commercial closed-source version as well, meaning you can opt for the closed –source when you want more privacy regarding the specifics of your application's database organization, and don't want to adopt that open-source approach. Due to its nature of being open-source, however, this means that it has a similar advantage that PostgreSQL has: a robust support system. This tells us that many experts in database system management are a part of the upkeep, meaning it is in great hands. Another advantage is that as a database system, it is very reliable. This reliability is another reason it is a popular choice for companies regarding data storage for applications. MySQL has a strong track record for effectively handling large amounts of user data. Another advantage is that MySQL is very feature-rich. It includes support for SQL, tools for ensuring proper security, effectively importing and exporting data, as well as much more. This is also shown when looking at its cross-platform capabilities. No matter what OS you are working on, MacOS, Windows, or Linux, you have access to all of the same features. This database system is also ACID compliant, which also adds support to its reputation of reliability.

MySQL is still a highly recommended database system, but it still has its disadvantages to consider. As feature-rich as MySQL is, it still lacks features that platforms like PostgreSQL have, such as support for JSON data types. This can lead to issues regarding reading and writing unstructured data. It also does not have proper debugging features, which can be a problem for beginners trying to debug database-related issues. Many MySQL developers recommend the use of third-party debuggers to use in tandem with the platform. They also recommend the use of third-party tools when trying to backup the database. It offers a great system of organized storage, but can easily cause issues regarding recovery of the database. Another disadvantage of MySQL is the response time of its servers. The database system is known to have a slow response time, which can cause a major communication issue if you happen to be collaborating with other developers on the same project. This can be highly frustrating, coupled with many of the other disadvantages listed previously.

Database Systems: Oracle

Oracle is an extremely popular closed-source multi-modal relational database management system, first developed in 1979 by Software Development Laboratories, which later changed to the Oracle Corporation, founded by Larry Ellison, Bob Miner, and Ed Oates. Oracle was created for the purpose of being a greater alternative to many of the current hierarchical database management system models out there at the time, tackling the limitations of those existing management and storage methods. Oracle is very commonly used as a database for data warehousing, online transaction processing, and mixed database workloads. It uses SQL for any of its database retrieval and updating, which is common amongst many of these relational database management systems. Oracle's development actually coincided with the development of SQL, so this makes much sense.

What leads to Oracle's popularity as a database management system is the strength of its many advantages. One major advantage is the robustness of its features, such as their data management tools. Oracle has a great team of developers that provide a wide range of tools for anything database related. Its feature strength also

allows it to support many different types of unstructured data like JSON and XML, spatial data, and structured data with SQL. Oracle also has a great reputation for being a reliable database management system. Many popular corporations and organizations employ the use of Oracle for their database management, such as Intel, JPMorgan Chase, NASA, and many more. Another Oracle advantage is its strong backup functionality. It offers automatic backups ranging from partial or full backups to single table restoration. Along with this comes great security, providing effective safeguards preventing unauthorized access to its databases. You can also choose to have encrypted backups as well, which is one way of showing its wide security options. Another great advantage is that Oracle offers incredible scalability and performance. Businesses can easily include additional hardware and server resources, providing extreme flexibility with over 100 platforms available. It also has a focus on optimizing performance, frequently increasing query time and operational speed. Oracle also makes the promise of ACID compliance, which leads to the nature of its reliability.

One disadvantage that Oracle has is the fact that it is a closed-source database management system. This brings with it the fact that it does not have as much of a robust support system than that of the previous systems in consideration. Oracle still has the incredible support of the team of developers that work for the Oracle Corporation, but that still pales in comparison to the sheer amount of people who are capable and able to support other open-source database management systems from the outside, no matter how skilled the developers that work on Oracle are. The cost of the system also contributes to its closed-source nature. It not being a free database management system prevents many smaller companies or users from benefiting, leading them to consider other free comparable systems like PostgreSQL or MySQL. Another disadvantage Oracle has is its sheer size. It can require a significant amount of resources to run when installed, which might lead to users requiring hardware upgrades before even implementing the system. This also relates to the higher system cost compared to free systems. One major disadvantage Oracle also has is the requirement of expertise regarding some of its major features. It is not very simple to use, meaning that a beginner database manager will have a very steep learning curve when being

introduced for the first time. This can lead to longer and more difficult learning periods when faced with more complex implementation.

After much consideration, we will be using PostgreSQL for the creation of our database, due to the extremely robust support system within the community, as well as the number of features it has compared to the other database systems being considered. Another major advantage is that it can provide high scalability, especially with complex data relationships, and still have great performance handling. You can see these comparisons easily in Figure 8 below, showing the benefits of PostgreSQL as a database system, with comparisons of the two other systems mentioned. These were major considerations when researching PostgreSQL and ultimately created more confidence in it's decision for our database management system. However, it helps to look at comparable database systems in order to understand the benefits and drawbacks of its implementation.

Figure 8: Database System Comparison Table

Database System	PostgreSQL	MySQL	Oracle Database
Open-Source	Yes	Yes	No
Cost	Free	Free	\$17,500 per Processor \$800 per User
Learning Curve	Intermediate	Relatively Simple	Relatively Difficult
Performance	High	Mid	High
Scalability	High	Limited	High

4.1.6 Backend Technology Research

After all of this consideration, we need a way for our PostgreSQL database to be connected to the rest of the project, for either gathering data to store or using certain relevant data in other parts of the project. This means that we need to employ backend technologies that really suit our needs; ones that can be easily understood and useful,

while simultaneously achieving the wants and requirements for our backend relations. Relevant technologies need to be looked at that conform to our requirements and will be considered. We will look at relevant technologies such as Python, Java, and PHP and determine which would be the best use of resources, as well as work best in tandem with the rest of our relevant project sections.

Programming Languages: Python

Python is a widely popular high-level coding language first developed in 1991 by Guido van Rossum, who started its development in the late 80s. Its initial purpose was to be a successor to the ABC programming language, emphasizing code readability and general-purpose coding. Python is popularly used in the machine-learning community, given its wide range of supporting machine learning libraries. It is also taught as an introductory coding language at a large number of learning institutions, such as UCF for example. Its simple to use syntax and easy readability make it an excellent candidate for this kind of role.

Python is a great language regarding a simple backend. It is an easy-to-use language, with a huge library of supporting functions we can use for any aspect of this project, especially for connecting each of the different parts together. Its simplicity makes it a very easy coding language to just pick up and use for anything coding related, even if you are more used to an entirely different type of coding language like C or HTML. It also has strong versatility regarding the amount of Python-supported tools out there, supporting different application types like machine learning, data science, artificial intelligence, and so much more. This is due to the extensive libraries and frameworks that are supported for this language. The language has a robust support system as well, with a large active community behind its upkeep.

Python as a backend language has its drawbacks as well. One drawback is that it is slower than compiled languages like Java, due to its nature as an interpreted language. This means that each line of code is executed one by one by the interpreter. Compiled languages, on the other hand, are converted into machine code before execution, making them faster by default. This makes it less of an attractive choice when dealing with large datasets or any other performance-critical applications.

Although, this disadvantage can be mitigated with select Python libraries like NumPy and Pandas. Another Python disadvantage is that it's less secure than other backend languages. The reason for this is that Python is a dynamically typed language, meaning that data types are determined at runtime over compile time. This leaves it vulnerable to injection or buffer overflow attacks. Another reason is due to Python's popularity and its wide array of third-party libraries. Insecure libraries can pose big security risks. Another disadvantage is Python's memory consumption and garbage collection. A lot of what makes Python a great language, its dynamic nature and ease of use, lead to high memory consumption. This can be a huge problem in resource-constrained environments, making it not the best language to use in imbedded systems or mobile environments. Its garbage collector is also not as efficient as Java's, for example. It may not release memory as quickly or effectively, leading to potential memory leaks or other issues regarding high memory usage.

Programming Languages: Java

Java is another widely popular high-level general-use coding language. Primarily an object-oriented programming language, it was first designed by James Gosling working at Sun Microsystems in 1995. Its purpose is to enable programmers to write once, run anywhere. This means that any compiled Java code can run on all platforms that already support java, without having to recompile on that platform. It's popularity extends within many web development and big data technology frameworks. It is also popularly taught as an introductory object-oriented programming language, due to its nature as an object-oriented programming language.

One advantage that Java has is that it is a relatively simple language to learn. Albeit Python is a much easier to start out with compared to most other high-level programming languages; Java is one of the more simpler ones to pick up and implement. This is due to its removal of complex features like pointers, structures, unions, etc. That languages like C and C++ implement. This leads to another Java advantage: security. Features like pointers give you direct access to memory location, which can bring about security risks like unauthorized memory access. Java handles all of the memory on its own, giving you minimal memory manipulation access. It also

employs certain object-oriented programming concepts like encapsulation, abstraction, inheritance, etc. for increased security. Another main advantage of Java is that it is platform-independent. This is due to its write once, run anywhere portability as mentioned earlier. To achieve this, Java compiled into bytecode then executed on a Java Virtual Machine (JVM), which is needed for any system with java capabilities. The JVM acts like a mediator between the Java software and the computer hardware.

Java also has a good amount of disadvantages as well. One disadvantage is that the language is slower than other popular high-level languages due to its JVM and other representational layers. This doesn't mean java is slow, however, it's still faster than Python due to its nature of being a compiled language. This just means there is always a layer between your code and the hardware, meaning that if you are writing an engine for real-time stock trading, or any latency-sensitive software, you may want to consider other choices. Another disadvantage is its memory consumption. Java handles a lot of its own memory allocation, which includes its robust garbage collector. This doesn't mean it takes up a light amount of memory though. The JVM uses a much larger amount of memory than you would think, which is the cost of the large convenience that is Java's very effective garbage collector. This is not a big deal for long-running backend systems, but runs into the same memory issues as Python when it comes to resource-scarce environments. Another large disadvantage is how verbose the syntax of Java is. This means that a simple java object includes a large pile of boilerplate like a class, getters, setters, a constructor, and more, even if the object stores minimal data. Languages like Python handle the same issue with much less boilerplate.

Programming Languages: PHP

PHP is a general-purpose scripting language created in 1995 by Rasmus Lerdorf. Its aim was to be geared towards web development, initially being written as a few CGI programs in C to maintain Lerdorf's own personal web page. From here he kept on adding functionality to work with web forms and database communication, naming it "Personal Home Page/Forms Interpreter" (PHP/FI). He released this version publicly to help with bug reporting, never intending for it to be a programming language. From here it started to grow organically though, consistently adding features until

eventually releasing the full language version PHP/FI 2 in 1997. What's unique about this language is that it grew organically, and wasn't initially designed to be a programming language. Nowadays it is popularly used as a web development backend for many technology stacks, such as the LAMP stack.

An advantage that PHP has is that it has major platform independence, being able to run on most operating systems such as Linux, Windows, MacOS, and UNIX. Because of this, developers can deploy PHP applications on any of these operating systems without having to make any major modifications. Another advantage is its integration capabilities. PHP is made to very easily integrate with many different web servers and protocols, as well as databases such as PostgreSQL, MySQL, Oracle, etc.. This makes it easy to implement with whatever database systems you are using, in this case PostgreSQL. PHP as a language has been largely adopted by the web development community, meaning a large amount of popular web applications are PHP-based. This means that implementing this as a backend can lead to growth opportunities elsewhere, as well as making you more familiar with other websites' backend functionalities. PHP is also an open-source language as well, which means it has an ample community of independent developers creating libraries and support systems to help its development.

One disadvantage that PHP has is its inconsistency regarding its design and syntax, due to the nature of its evolution as a programming language over the years. This brings more difficulty to learning the language itself, and can make it feel less cohesive compared to other languages like Python or Java. This leads into another disadvantage with the language regarding more advanced concepts, such as object-oriented programming or advanced database interactions. PHP has a steep learning curve regarding advanced topics if you are a beginner to the language, and is less suited for learning compared to other languages. PHP poses many security risks as well. Due to its open-source nature, PHP applications can be highly susceptible to SQL injection and cross-site scripting attacks. Another large disadvantage is its issues with dependency management. PHP can have a large amount of conflicting dependencies

and outdated libraries, which means having to manually deal with these issues. This can be very time-consuming when ensuring stability and reliability with PHP.

After much consideration, we have decided to go with Python as our primary backend technology. With its massive popularity, it has a ton of support regarding web development. This is to say, the amount of libraries that are tailored specifically to what we are trying to achieve is astounding, such as psycopg2 and Flask. The language is comparatively extremely easy to use, having such user-friendly syntax as well as easy access to the aforementioned libraries. It is easy to implement with other technologies as well, providing little to no issue when accessing or acquiescing the relevant data for our PostgreSQL database. As you can see in Figure 9 below, these comparisons are laid out and easily show the advantages of each language. All of these advantages lead Python to be a great choice, allowing us to extend our backend capabilities as much as possible, while still having the support needed to achieve this extension.

Figure 9: Backend Technology Comparison Table

Name	Python	Java	PHP
Performance	Mid / Interpreted / Mitigated w/ Libraries	Relatively High / Compiled	Mid / Interpreted
Simplicity	Extremely Simple / Beginner Friendly	Relatively Simple / Beginner Friendly	Not Simple
Memory Management	High Consumption	High Consumption	Mid consumption
Community Support	Extremely High / Plethora of Libraries	Relatively High	Relatively Mid / Lesser Support

PyCharm

PyCharm will be the primary consideration regarding the IDE for our Python backend development. It is an incredible IDE, created by JetBrains specifically for developing with the Python language, boasting a robust support system for current

Python libraries, as well as any Python-related tool out there. A major benefit is that it is cross-platform, which makes switching work environments to continue development a simple process. PyCharm also has a strong debugger built into the IDE already, which gets rid of the need for an external debugger. Along with a debugger, it offers intelligent code completion that suggests variable names, code snippets, etc. Which saves a tremendous amount of time and allows me to focus more on logic than repetitive tasks.

DBeaver

DBeaver is a popular free universal graphical user interface (GUI) for database management. Most developers and database management specialists use this software tool to manage a variety of databases, including SQL and NoSQL database types. A key feature of this tool is its robust SQL Editor. It offers a simple interface for executing SQL queries with ease. One incredible feature is its intelligent code completion. As you type, it autocompletes with suggestions like column names, table names, etc., making the workload lighter and flow more easily. DBeaver also provides an extensive Data Viewer, providing a simple way to view query results, as well as an overall view of its tables in a grid system. Another key feature is its Entity-relationship Diagrams. This allows us to see a visual relationship regarding each of the tables and their relationships. We will be using this to access our PostgreSQL database.

4.1.7 Database Design

With all of these factors and technologies in mind, we can now start to design and create the actual database. As we decided previously, this will be a relational database using PostgreSQL as our database management system. We want each transaction to be stored as a complete record in our database. This means containing every aspect of the items being stored:

- ID
- Timestamp
- Stable weight
- License plate
- Cargo classification

- Method of weight capture
- Direction of the scale
- Photo of the vehicle.

Each vehicle will be a separate transaction, stored with its own ID as an integer. Each ID will be unique, being connected to its relevant information regarding the vehicle being weighed. It will be connected to the proper stable weight stored as an integer, along with the vehicle's license plate stored as a string, cargo classification stored as a string, timestamp stored as a string, method of capturing stored as a string, and the direction of the weight scale stored as a string. All of this will then be posted onto an image of the vehicle and stored as well. For image storage, once we have a way to create the image itself, we will have an official decision made for storing that image. This information will be stored in a relatively simple data table, as there does not seem to be a need for a complex database relationship system regarding each vehicle customer.

4.1.8 API Design

To decouple the hardware communication logic backend system, we designed a lightweight REST API based module. This middleware layer manages the continuous, low-level data stream from the physical scale and exposes the latest stable data via standard HTTP endpoints for easy access.

System Architecture: The API is built using Python's Flask library. It utilizes a threaded architecture to handle blocking hardware I/O without freezing the web server:

- Main Thread (Flask): Handles incoming HTTP requests from the frontend or database logger.
- Background Thread: Managed by the WeightScale class, this thread maintains a persistent TCP socket connection to the scale. It continuously buffers incoming bytes, parses them by newline characters (\n), and updates a thread-safe variable with the latest reading.

Class Structure: The core logic/cache consists of the WeightScale class, which handles:

- Socket Management: Establishes and maintains the socket.AF_INET (IPv4) and socket.SOCK_STREAM (TCP) connection.
- Buffering: Reads data in 4096-byte chunks and reassembles fragmented messages to ensure complete weight readings are processed and stored for API use.
- Concurrency: Uses threading.Lock() to prevent race conditions when the background thread updates the weight and the main thread tries to read it.

API Endpoints: The system exposes the following JSON endpoints:

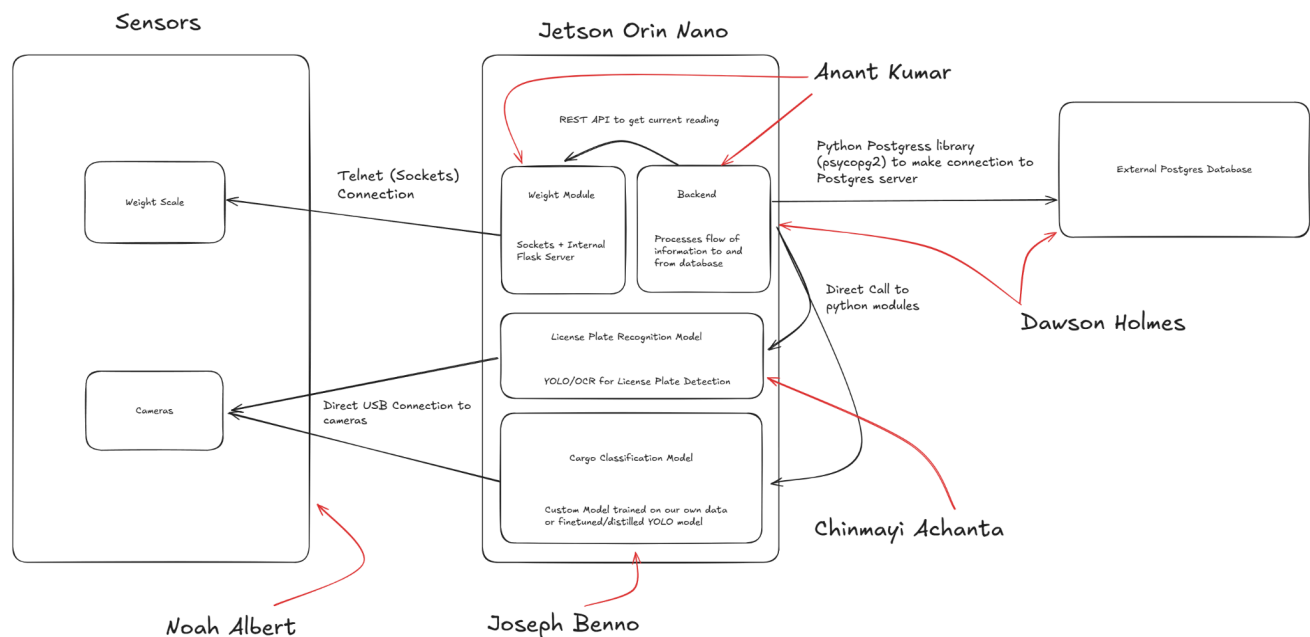
- GET /reading
 - *Purpose*: Returns the latest stable weight reading.
 - *Data*: JSON object containing the raw weight value, a timestamp of the last update, and connection status.
 - *Usage*: Polled by the backend to get the latest weight data.
- GET /status
 - *Purpose*: Provides diagnostic connection information.
 - *Data*: Boolean flags for connected and running.
 - *Usage*: Used to verify if the background thread is still connected to the weight scale
- GET /health
 - *Purpose*: Simple liveness check.
 - *Data*: {'status': 'ok'}.
 - *Usage*: Used to check for current health of module
- GET /
 - *Purpose*: Documentation.
 - *Data*: A list of all available endpoints and their descriptions.

4.2 System Diagrams

4.2.1 High-Level System Block Diagram

Figure 10 illustrates the overall system architecture and shows how data flows from the sensors to the final database record. The weight scale connects to the Jetson Orin Nano through a Telnet (sockets) interface, while the cameras connect directly via USB. On the Jetson, three major modules operate together: the Weight Module, the License Plate Recognition pipeline (YOLO + OCR), and the Cargo Classification model. These modules communicate with the backend service, which processes stabilized readings and prepares structured transaction data. The backend then connects to the external PostgreSQL database, where all weigh-in/weigh-out events are stored. The diagram also shows how team responsibilities are divided across these components. Overall, Figure 10 provides a clear visual summary of the system's end-to-end workflow and the interactions between hardware, software, and data handling layers.

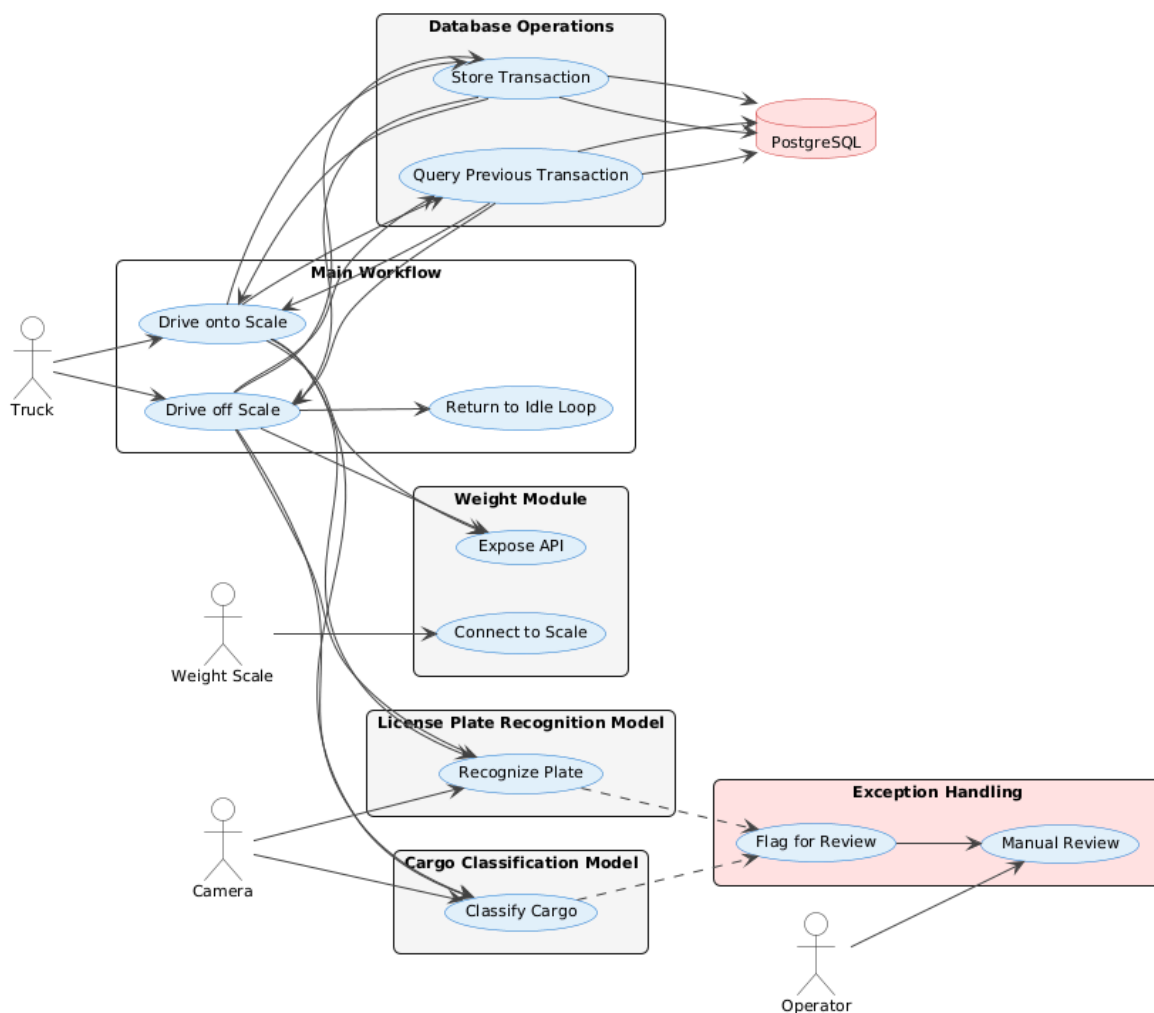
Figure 10: System Block Diagram



4.2.2 Use-Case Diagram

Figure 11 illustrates the main use cases within the automated weigh-in/weigh-out system. It highlights how external actors (truck, weight scale, camera, and operator) interact with the core modules responsible for weight capture, license plate recognition, cargo classification, and database operations. As the truck moves onto and off the scale, the system processes weight and visual data, identifies the correct transaction, and stores results in PostgreSQL. The diagram also shows the exception flow, where uncertain detections are flagged for manual operator review. Overall, it provides a concise view of how each subsystem contributes to the full workflow.

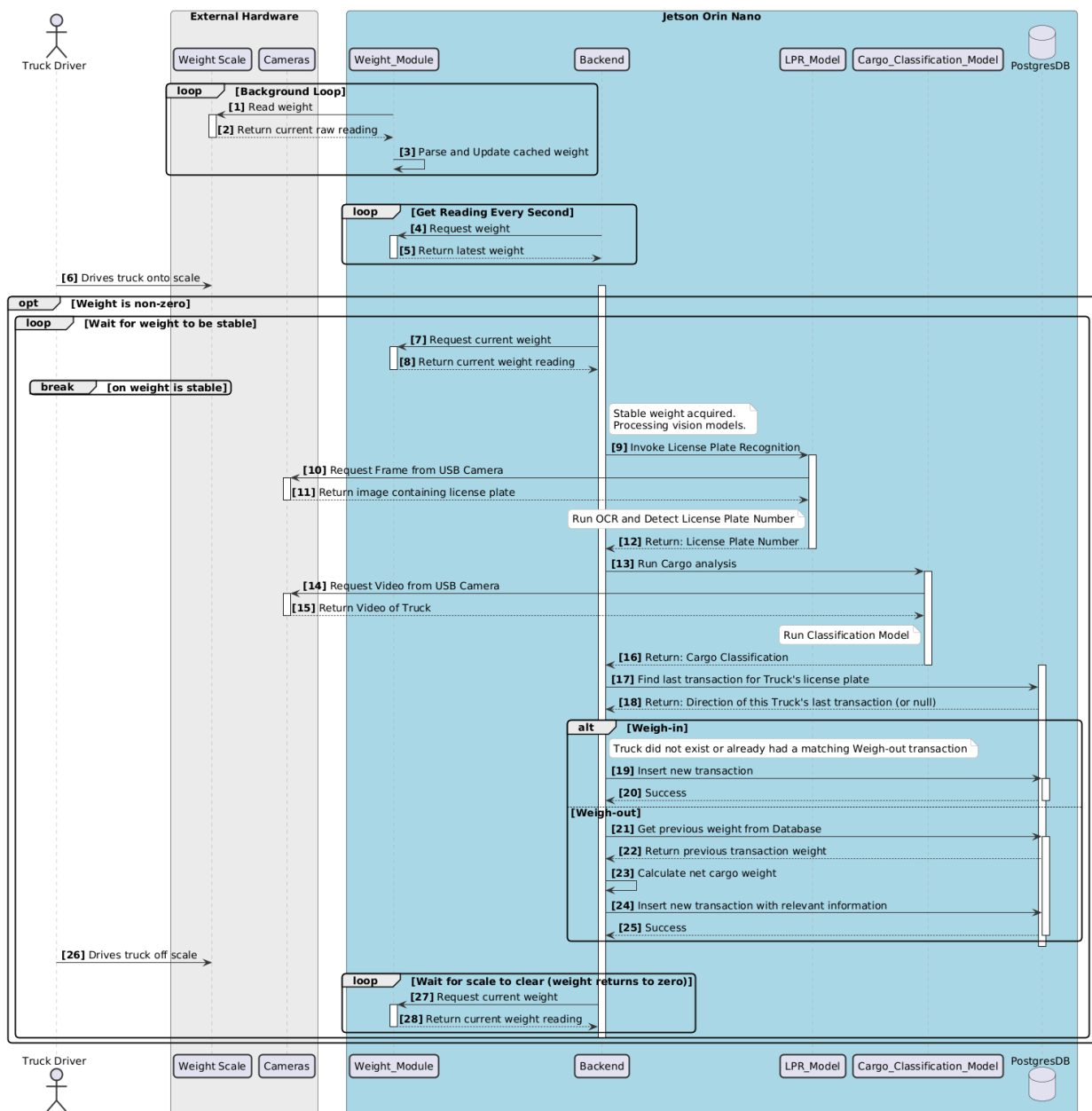
Figure 11: Use-Case Diagram



4.2.3 Activity and Sequence Diagrams

Figure 12 illustrates the full sequence of the weigh-in/weigh-out process, showing how the Jetson Nano reads weight data, detects stabilization, triggers license plate recognition and cargo classification, and then stores the appropriate transaction in the PostgreSQL database. The diagram captures the end-to-end flow from a truck entering the scale to the system returning to its idle state once the scale clears.

Figure 12: Weigh-In/Weigh-Out Sequence Diagram



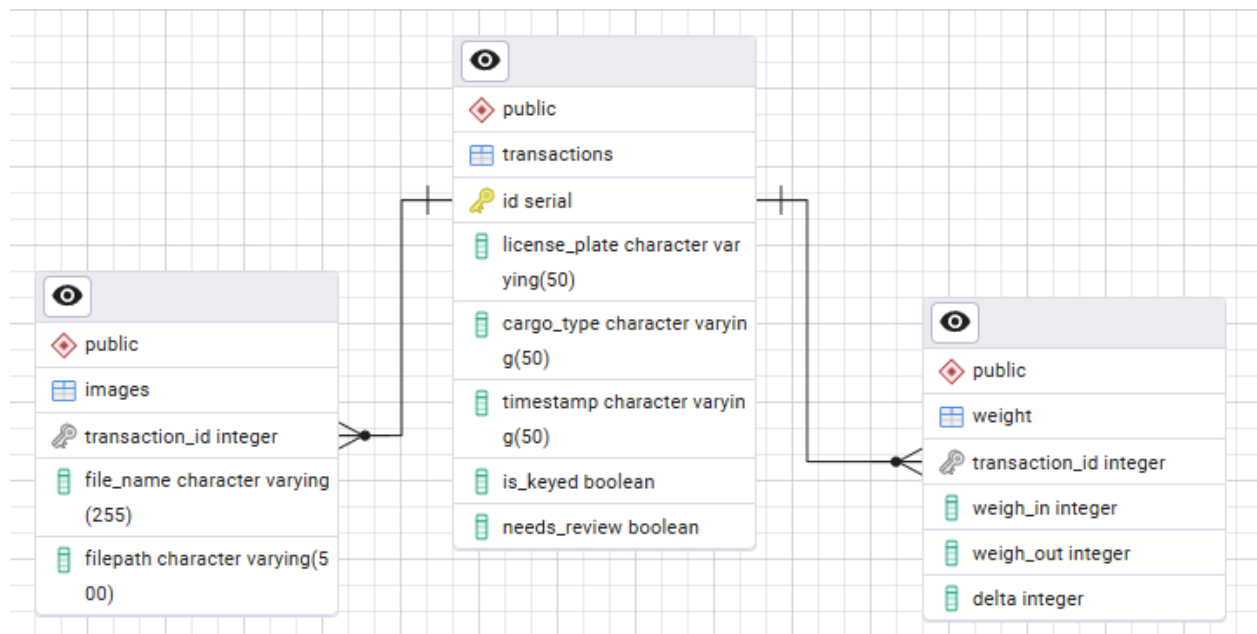
4.2.4 Database Entity Relationship Diagram (ERD)

Our Vehicle Weight System uses three PostgreSQL tables, our primary table labeled transactions, with secondary tables labeled weight and images, shown in Figure 13. Because each weigh-in or weigh-out event is fully self-contained, no additional relational tables are needed. All important information for identifying a vehicle, determining cargo weight, and validating system output can be stored in one record per event, keeping the database simple, efficient, and easy to maintain which is ideal for an embedded Jetson-based system.

The transactions table stores each event using a unique serial ID, along with the detected license plate, predicted cargo type and the event timestamp, as well as two boolean variables `is_keyed` and `needs_review`. This is our primary key table, holding the main truck identification information. The weight table stores all of the relevant weight information, such as the in-bound weight, out-bound weight, and a delta variable between both of them. Our final table keeps track of how we store the main image of the truck itself. This stores the name of the image file, as well as the filepath for the actual image.

Our database is using the transactions table as our primary table, using the `id` variable as the primary key, and using the `id` variable in each subsequent table as the foreign key pointing to the transactions table. This streamlined, multi-table structure minimizes complexity while still capturing all essential data, supporting fast queries, low overhead, and reliable transaction tracking for the weigh-in/weigh-out workflow.

Figure 13: Transactions Entity Relationship Diagram



4.3 Subsystem Designs

4.3.1 Machine Learning & Computer Vision

4.3.1.1 YOLO Object Detection Pipeline

The YOLO-based object detection subsystem is responsible for localizing the license plate within each captured frame. While Section 4.1 discussed the reasoning behind selecting YOLOv8n, this section focuses on how the detector operates within our final system design.

System Inputs and Preprocessing

The YOLO pipeline receives a single RGB frame directly from the USB camera. Before inference, the frame is resized to YOLO's expected input resolution (typically 640×640) and normalized. This resizing ensures consistent inference performance on the Jetson Nano and minimizes GPU memory usage.

Inference and Model Execution

The detector runs as an optimized ONNX or TensorRT engine on the Jetson Nano to ensure low-latency inference. YOLO outputs a set of bounding boxes, each with an associated confidence score and class label. The model is configured to detect a single custom class ("license_plate"), which simplifies post-processing and eliminates unnecessary filtering.

Post-Processing and Bounding Box Selection

After inference, non-maximum suppression (NMS) is applied to remove redundant or overlapping boxes. The system selects the bounding box with the highest confidence score. If no detection exceeds the minimum threshold, the LPR pipeline flags the frame for re-capture or passes it to an error-handling routine.

Output to OCR Subsystem

Once a valid plate is detected, the bounding box coordinates are used to crop the plate region from the full frame. This cropped region is passed to the OCR subsystem. The YOLO stage also returns metadata (confidence scores, bounding box size, and plate location) which can optionally be logged for debugging or evaluation.

4.3.1.2 OCR Text Extraction (EasyOCR)

The OCR subsystem receives the cropped license plate image and extracts readable text. This component converts visual data into the alphanumeric identifier used throughout the rest of the system.

Input and Preprocessing

The OCR module receives a small cropped plate region (typically 120 - 200px in height). Before recognition, we apply lightweight preprocessing to enhance clarity: grayscale conversion, contrast adjustment, sharpening, and optional de-skewing. These operations improve robustness under glare, uneven lighting, or dirty plates without introducing significant latency.

OCR Inference

EasyOCR runs on the Jetson Nano using GPU acceleration when available. The engine outputs character predictions along with per-character confidence values. Its deep neural architecture allows it to interpret partially obstructed or low-contrast text better than traditional OCR engines.

Post-Processing and Validation

After extraction, the raw OCR output is passed through a validation layer that:

- removes invalid characters
- normalizes spacing
- enforces expected plate patterns (alphanumeric regex)
- checks confidence thresholds

If the recognized plate falls below the required confidence score, the system flags the event for manual review or applies fallback strategies (e.g., attempting a second pass with adjusted preprocessing).

Output to Backend

The final processed plate string is returned as a clean, validated identifier. This value is packaged with weight, cargo classification output, and timestamp and then passed to the transaction builder for insertion into the PostgreSQL database.

4.3.1.3 Pre-Processing and Post-Processing Steps

For this project, our dataset comes from video recordings taken directly from the weight-in and weight-out camera at the facility. Since this specific camera has its own angle, height, and lighting conditions, there are no public datasets that match what we need. Because of that, we have to build our own dataset by breaking down the raw video footage into clean, consistent images. These pre-processing steps are shared between both the cargo classification model and the license plate recognition model, since both depend on clear images from the same source.

The first step is to extract frames from the video. We use Python scripts with libraries like OpenCV to load the video file and sample frames at a fixed time interval. We do not take every frame because that would create an extremely large dataset with too many repeated images. Sampling at a steady interval keeps the dataset manageable while still capturing changes in lighting, motion, and visibility of the truck bed and the license plate.

After we extract the frames, we go through and remove the ones that are not useful. This includes frames that are blurry, too dark, or taken at moments when the truck or the plate cannot be seen clearly. Some frames also show the truck only partially entering or leaving the scene, so we remove those as well. Cleaning out these low-quality frames helps make sure that both models train on images that actually contain the information we need.

Next, we crop each image to the important regions. For cargo classification, we crop around the truck's load area, and for license plates we crop the part of the frame where plates usually appear. The original camera view includes a lot of unhelpful background, such as the sky, buildings, or ground. Cropping removes these distractions so the models can focus on the features that matter. We use simple Python scripts with OpenCV functions to define and apply these crop boxes consistently across all images.

Once the images are cleaned and cropped, we label them. Cargo images are sorted into the correct material type, and license plate images are matched with the correct text. Since real video footage does not give us a perfect balance of each category, we apply data augmentation to expand the smaller classes and improve variety. Adjusting brightness, contrast, color, rotation, and adding small amounts of noise helps the models learn to handle different lighting situations and natural variation in the video.

After labeling and augmentation, we resize the images and normalize them so they match the input format required by the models we plan to use. Doing this ensures that training conditions match the images the edge device will see during real use. We

also use the same preprocessing code during training and during live inference so there is no mismatch.

Finally, to evaluate how well the full pipeline performs, we set aside part of the original video as a separate test video. We process it the same way, but we do not use it for training. This gives us a realistic way to test the entire system on completely unseen data, which is important to confirm that the models generalize well and behave correctly under real operating conditions.

4.3.1.4 Cargo Classification Model

The cargo classification system works as a full pipeline running on an edge device, the Jetson Orin Nano, and saves all results in a PostgreSQL database. Its main purpose is to automatically identify the type of load each truck is carrying. The current labels include general waste, construction debris, paper, and wood.

Image capture is directly linked to the truck scale. When a truck activates the scale, the system records the initial weight readings and waits for the value to stabilize. Once the scale reading becomes stable, the system captures a single image of the truck's load area. Although taking multiple images can sometimes help with motion blur or lighting issues, the current system captures only one image after stabilization to keep processing fast and lightweight. Even with a single image, the pipeline is designed to perform reliably under realistic conditions where image quality may not always be ideal.

The captured image is then sent to the Jetson for preprocessing and model inference. During preprocessing, the image is resized to a fixed 224×224 resolution, converted to RGB if needed, and normalized using the standard mean and standard-deviation values expected by the classification model. This prepares the image so it matches the format used during the model's training. The main model used is EfficientNet-B0 because it offers the best accuracy for its size, with MobileNetV3-Small kept as a lighter backup option.

For each truck event, the model processes the single captured image and produces a label prediction. The predicted label, its confidence score, and the

timestamp are then stored in a PostgreSQL table for later review and system integration.

Performance Evaluation Metrics

To understand how well each model performs and whether it fits the needs of our system, we use a set of evaluation metrics that capture different aspects of accuracy and class reliability. These metrics help us judge not only how often the model is correct, but also how well it separates similar classes and how consistent it is across all categories.

Top 1 accuracy is the main measure we look at. It simply checks how often the model's highest prediction matches the true label. Since our goal is to automatically classify each truck load without human correction, this value gives the clearest indication of how dependable the model will be during real operation.

Top 5 accuracy gives us deeper insight into how well the model understands the differences between categories. Even though we do not have a very large number of material types, Top 5 accuracy still matters because it shows how the model ranks the other possible classes. For example, if the correct label does not appear as the first guess but shows up as the second or third guess, that tells us the model is at least looking in the right direction.

This is especially useful when classes share similar textures or colors. For instance, recycled paper and cardboard often resemble each other, and construction debris can sometimes look similar to mixed waste or wood. A strong Top 5 score shows that the model is able to place the correct category near the top of the list, even if it does not rank it first. In other words, it helps us see whether the model truly understands the structure of the classes and how distinct they appear to the network.

For more detailed analysis, we also use precision. Precision checks how often the model is correct when it predicts a specific class. If one category is predicted too often, precision will drop, which can help us identify classes that are overfit.

Recall looks at the opposite side: how many true examples of a class the model actually finds. Low recall means the model is failing to detect many real examples, which can create missing or incomplete records.

Finally, the F1 score combines precision and recall into a single number. This metric is especially helpful for datasets that are not perfectly balanced, which is common in real waste-handling environments. Some materials appear far more often than others, so accuracy alone may not give a complete picture. The F1 score helps us understand how well the model performs across all classes, not just the most common ones.

4.3.2 Sensors and Scale System

4.3.2.1 Scale Simulator Description

The scale simulator is a resource provided to us by our sponsors, specifically designed by Scott Piersall. The simulator simulates multiple types of signals from any kind of commonly used weigh house configuration. As we do not have access to an actual weigh house or scale to work with, this program allows us to build software that accepts inputs in the same form and format as what we will see in practice.

There are many differing configurations that weigh houses may have their scales in and the simulator allows us to account for each of these cases, and ensure our software will be compliant for any weigh house which our solution will be installed in. The simulator offers options for one or two scales, scale stability and motion detection, and timestamping of measurements. In addition to these configuration options, we can also simulate a tare operation or manually keying in a weight. There are also options to simulate a truck driving onto and off of the scales, that will gradually increase or decrease the weights respectively until arriving and stabilizing at a certain weight. If these readings are too regular or repetitive, the simulator includes a slider and jitter function to manually set weights or introduce variance.

4.3.2.2 Weight Stability Detection Algorithm

The weight stability algorithm will be constantly storing the previously read weight, and comparing the current reading to this. When the readings are the same, a timer will be started. If any different measurement is read, the timer will be set to zero and this process will start over. If the weights are the same for enough duration, the weight can be considered stable and a measurement can be taken.

In the case of motion detection scales, working with this becomes easy. The methods detecting a stable weight can wait to be started until the scale has stopped detecting motion. Of course, in all systems these stability detection methods will not run if the weight of the scale is zero.

In the case of systems which include time stamps alongside the readings from the scale, the timing for a weight to be defined as stable becomes incredibly simple. We will read the time when a new value is transmitted, and then continue to read the weight and time until the weight reads the same after a certain time elapsed. In all other systems which do not include a timestamp, we will need to start a timer when new weights are read, and then only stop the timer when the weight is legally compliant, or restart the timer when a new weight is read.

4.3.3 Backend Integration

4.3.3.2 API Communication and WebSocket Setup

While initial research considered WebSockets for the scale communication, the final implementation uses a TCP Socket approach (`socket.AF_INET`, `socket.SOCK_STREAM`) to interface with the network weight scale.

Socket Configuration: The weight module creates a persistent client socket connection managed by a background thread, constantly updating the cached values as new data comes in. This setup is important to handle the continuous data stream that the scale emits without blocking the main application waiting for the next value

- **Connection Parameters:** The socket is configured with a 10-second connection timeout and a 5-second read timeout to detect hardware/network failures that could lead to inaccurate measurements.
- **Buffering Strategy:** Data is read in 4096-byte chunks and appended to a rolling buffer. A delimiter-based parsing logic splits this buffer by newline characters (\n) to extract individual weight readings as it arrives, ensuring that fragmented packets do not result in partial data reads.
- **Thread Safety:** A `threading.Lock()` mechanism protects the `current_reading` variable. This allows the background thread to write new data and the Flask API thread to read data simultaneously without race conditions.

4.3.3.3 Authentication and Security Controls

Since the whole system runs directly on the Jetson Nano and does not connect to any public networks, the security needs for this project stay fairly basic. The only authentication in place is the username and password required to access the PostgreSQL database, which limits who can view or change stored records. Every other part of the system, including the camera, model pipeline, scale connection and local API, operates only inside the device and cannot be reached from outside.

Because the setup is fully local, we did not add extra security features such as token systems or encrypted access layers. If the project grows into a multi-site system or becomes accessible over a wider network, stronger security controls may be needed. For now, the existing local database login is enough to protect the system in its current environment.

4.3.4 Hardware Deployment

4.3.4.1 Jetson Nano Configuration

To ensure a reliable and reproducible deployment environment that can be copied to multiple Nanos easily, we established a configuration protocol for the device:

1. Python Environment: We utilize a virtual environment (venv) to isolate project dependencies from the system-level packages and allow us to easily update or freeze package versions.
2. Service Management: A systemd service file was created for the entire system, meaning the system can easily recover and reboot from a power outage or other failure that brings the device offline.

4.3.4.2 Camera Interface and Capture Logic

After the weight on the scale has been detected as stable, the camera can be enabled and begin recording. The camera will capture an image of the rear of the truck and use OCR to read the license plate. This license-plate-based identification can be linked to the measurement taken from the scale, and uploaded to the database.

The camera will connect to the Jetson Ohin Nano via a USB, and should be positioned in a standard manner in order to capture the rear of the vehicle being weighed.

4.3.4.3 System Initialization and Remote Updates

To meet the sponsor's requirement for long-term maintainability without physical site visits, the deployment architecture reuses the secure connectivity layer used during the development phase.

Initialization and Lifecycle Management The application lifecycle is managed by the Linux systemd init system. A dedicated service allows for simple automatic startup sequence:

- Auto-Start: The service is enabled to launch immediately upon boot, allowing the system to recover automatically after power outages.
- Restart Policy: We configured Restart=always and RestartSec=5 to ensure that if the Python process crashes due to an unhandled exception, the OS will automatically attempt to revive the service without human intervention.
- Logging: All standard output and error logs are redirected to journalctl, allowing administrators to review historical performance and error traces remotely.

Remote Update Strategy using the same Tailscale VPN configured during development, allowing administrators to perform over-the-air (OTA) updates to the software stack securely:

1. **Secure Tunneling:** The administrator connects their local machine to the VPN, allowing for direct SSH access to the Jetson Nano. This bypasses the need for complex port forwarding on the facility's router and keeps the device invisible to the public internet.
2. **Code Deployment:** Updates are deployed by pulling the latest release from the version control system directly onto the device.
3. **Dependency Management:** Python package updates are applied within the isolated virtual environment (venv) using pip, ensuring that updates to the application do not conflict with system packages.
4. **Service Reload:** Once the code and dependencies are updated, the systemd service is reloaded to apply the changes with minimal downtime.

5. Build, Prototype, and Test Plan

5.1 Build Overview

5.1.1 Development Environment Setup

Our development environment for this project is centered around the Jetson Orin Nano Developer Kit from NVIDIA, which hosts and acts as our core computing platform. In order to enable development, we have also added remote infrastructure such as VPNs and SSH servers to allow our team to develop without having direct access to the device.

For our VPN solution, we chose to use Tailscale, a simple WireGuard Based VPN solution to allow us to easily connect to and develop on our Jetson Nano. We also went through a few other solutions outlined below but decided against them in the end.

1. Direct Port Forwarding/Other direct tunneling: Would provide a simpler access than a VPN solution without much hassle past the initial setup. However, it would also expose the Jetson Nano to the open internet and could create security risks if not properly configured and protected. We ended up deciding against this, especially when considering existing code from other students existing on the machine already.
2. Traditional VPN: We also considered setting up a traditional VPN using OpenVPN or WireGuard. However, this would still require a central VPN server such as a VPS to proxy traffic to the Jetson Nano.
3. ZeroTier: We also considered more advanced configurations such as ZeroTier, however, we found this to be more complex than our project required.

Considering all this information we decided to take the existing VPN approach. With Tailscale, we would be able to, with some configuration on each group member's end, connect to the Jetson Nano easily.

With how to make the connection decided, we also had to choose what we would make the connection with. This meant deciding between two main choices:

1. SSH: The simple, but limited approach. Would allow us to easily connect to the Jetson Nano and gain terminal access to run commands or configure our environment. However, this would limit our ability to run GUI apps that we may need such as our database connection software or our scale simulators provided to us by our sponsors.
2. RDP: Remote Desktop Protocol, the advanced by full access approach. Would allow us full desktop access to the Jetson Nano, allowing us to easily run GUI apps or configure the device with a GUI. However, this would require a more advanced setup on both the server and client side, with us needing to download additional RDP apps to access the Nano.

In the end, we decided to go with the SSH server approach as it required a very simple initial setup. It would also mean that we could set up RDP access later down the road if need be, using the limited access to install the necessary software.

5.1.2 Hardware Platform Specifications

The hardware platform for our system is centered around the NVIDIA Jetson Nano, which serves as the primary compute unit responsible for running object detection, OCR inference, and all real-time processing required during weigh-in and weigh-out events. The Jetson Nano provides a quad-core ARM CPU and a 128-core NVIDIA Maxwell GPU, enabling efficient deep-learning inference on the edge while maintaining a compact power footprint suitable for continuous operation at a landfill site. Its compatibility with TensorRT, ONNX Runtime, and PyTorch makes it an ideal platform for deploying lightweight versions of YOLO and EasyOCR optimized for embedded systems.

A high-resolution USB camera connects directly to the Jetson Nano and provides continuous video input for license plate detection and cargo classification. This camera serves as the system's primary visual sensor, capturing frames immediately after the scale stabilizes. Its mounting position can be adjusted to ensure a consistent view of the license plate regardless of vehicle type or environmental conditions, and its plug-and-play interface simplifies integration with the Jetson during early development.

To replicate real operational environments, the sponsor has also provided a load-scale simulator that mimics the behavior of an industrial weigh station. The simulator produces real-time weight readings that allow us to test stabilization detection logic, validate the timing between scale events and image capture, and confirm that the backend processes transactions correctly from start to finish. While we continue to wait for access to a real physical scale, the simulator allows us to perform meaningful development and system testing.

Together, the Jetson Nano, USB camera, and load-scale simulator create a complete edge-processing environment that mirrors the workflow of the final deployed system. This setup allows us to test model performance, data flow, and system timing

with high fidelity, ensuring that the transition to real hardware later in the project is as smooth as possible.

5.1.3 Software Stack and Dependencies

Our software stack is intentionally kept simple because the project does not require a full frontend or a large backend framework. Most of the system runs directly on the Jetson Nano, and almost all of our logic is written in Python, which we already use for the machine learning and computer vision parts of the project. This keeps the system lightweight, easier to manage, and consistent across all components. While we could have switched to a lower-level language like C, C++, or Go for performance, Python remained the best choice for the team because it allows faster development, easier testing, and better integration with the tools we already rely on.

The hardware communication layer uses Python's built-in socket library to talk to the weight scale. The Jetson Nano creates a direct IPv4 TCP connection to either the real scale or the scale simulator. Once connected, the scale sends a constant stream of raw weight values. The software listens for new readings at a set interval, usually around one hundred milliseconds, although this can be adjusted if needed. A parsing module then extracts useful information from the incoming messages and formats it into a clear JSON object so the rest of the system can read it without worrying about the raw data format.

After parsing these readings, the stabilization algorithm checks how the values change over time to decide when the truck is fully on the scale and the weight has stopped moving. When the weight stays steady long enough, the system marks it as the final value. This data is then shared through a small Flask web service that exposes the most recent stable reading through a simple REST API. This lets the database logic and machine learning pipeline request the latest weight without managing the socket connection themselves.

The camera interface follows a similar approach. The Jetson Nano connects to the camera through a standard USB connection, keeping everything local and removing

the need for network-based video streaming. The captured frames are processed directly by the machine learning models responsible for license plate recognition and cargo classification.

Most of the machine learning workflow, including training, evaluation, dataset handling, and model loading, is built around PyTorch. This includes downloading or loading pre-trained models, running inference on captured frames, and fine-tuning models when needed. Using PyTorch keeps the ML pipeline flexible and lets us reuse well-supported tools such as torchvision, pretrained backbones, and existing evaluation utilities, all of which fit naturally into the Python environment we are already using.

Other project components, including the database logic and integration code, are also written in Python. Using the same ecosystem across the entire project makes development smoother and helps the team debug and test the system more easily. Even though each part has its own role, keeping everything inside one language and toolset allows the full system to work together with fewer compatibility issues.

For overall project management, we use Jira to track tasks, organize sprints, and coordinate work across the different subsystems.

5.2 Test Strategy and Results

5.2.1 Unit Testing (Individual Subsystems)

The unit testing phase validates that each individual subsystem functions as it should, outputting the correct information when passed with test data

- **Scale Connectivity:** Unit tests for the WeightScale Python class ensure it can handle connection drops and reconnect logic without crashing the main application if the weight scale ever happens to lose connection. Using the simulator, we will verify that the parsing logic correctly identifies stability markers in the data stream, testing edge cases where the stability is just under the threshold.
- **License Plate Recognition (LPR):** Initial testing will involve running our YOLOv8 model against a standard open-source dataset (like OpenALPR) to establish a

baseline accuracy. This ensures the model architecture is sound before we begin fine-tuning it with site-specific data. This will also be extended to ensure it correctly reads actual examples given to us by our sponsor.

- **Cargo Identification:** Testing for this module will consist of passing preexisting videos to the classification model and measuring accuracy and speed to ensure functionality when deployed.

5.2.2 Integration Testing (End-to-End Pipeline)

The integration testing phase validates that the different modules can communicate effectively between the different parts of the system, ensuring the flow of information from the truck to the database.

- **Data Flow Verification:** We intend to test the connection between the subsystems. Specifically, we need to verify that a "stable weight" signal from the Scale API successfully triggers the camera capture script.
- **Database Logging:** We will perform tests to ensure that when all three data points (weight, plate, cargo) are ready, they are correctly bundled into a single SQL transaction with all other necessary information and properly stored in the database.

5.2.3 Simulation Testing with Synthetic Data

Our sponsor provided us with scale simulator software that emulates the telnet protocol output of industrial weight scales the final project will see when deployed to actual weight stations. This simulator enables development and testing without requiring access to actual scale hardware.

The simulator is packaged as a single Windows executable (.exe) file we can run on our local machines to simulate a weight scale on the local network. It provides us with testing capabilities for various real-world weigh station configurations we may encounter during deployment. The simulator allows us to develop and validate our weight reading logic, stability detection algorithms, and data parsing functions before connecting to actual industrial scale hardware.

Simulator Capabilities:

- Protocol Emulation: Accurately emulates industrial scale telnet protocol output on a port of our choosing
- Network Broadcasting: Broadcasts weight readings continuously over a set port
- Multiple Scale Configurations: Supports both single-scale and dual-scale (ingress/egress) setups, as different weight stations may have only one or two scales
- Motion Simulation: Simulates realistic truck behavior including pulling onto scale, stabilization period, and driving off, allowing us to tune our reading to take only the stable reading
- Data Format Variations: Supports multiple output formats including timestamped data and motion indications, allowing us to calibrate and properly test our parsing logic on different configurations that we will see

5.2.4 Performance Testing on Jetson Nano

Since we are deploying to an embedded device (NVIDIA Jetson Orin Nano), we must ensure our software does not exceed the hardware's thermal or memory limits.

- Resource Monitoring: We plan to use the jtop (Jetson Stats) utility to monitor CPU, GPU, and RAM usage while the application is running. This will help us identify any memory leaks or processes that are consuming too much power.
- Thermal Benchmarking: We will run the system for extended periods of time in varying conditions to ensure the device stays within safe operating temperatures, which is critical for a system that could be installed outdoors or on edge scenarios.
- Inference Speed: We will establish a baseline for how many frames per second (FPS) the system can process. If the inference speed is too slow for real-time usage, we will investigate optimization techniques like TensorRT in the future.

5.3 Performance Metrics

5.3.1 YOLO Detection Accuracy and Inference Time

The performance of the license plate detection stage will be evaluated using two primary criteria: detection accuracy and inference time.

Detection Accuracy refers to how consistently the YOLO model correctly identifies and localizes the license plate within an image. This is typically measured using metrics such as:

- Precision - the percentage of detections that are correct.
- Recall - the percentage of actual license plates correctly detected.
- mAP (mean Average Precision) - a standardized metric that evaluates detection performance at various IoU (Intersection over Union) thresholds.

These metrics ensure that the detector is both reliable and robust across different lighting conditions, distances, and angles. High detection accuracy is critical because any missed or poorly localized plates will directly affect the downstream OCR results.

Inference Time measures how long it takes the model to process a single image. On an embedded device such as the Jetson Nano, this becomes a key factor in determining whether we can operate in real time. Our goal is to maintain inference times low enough that the detection pipeline runs smoothly without delaying the weigh-in/weigh-out workflow.

When hardware testing begins, we will log both accuracy and inference speed across representative samples, then use these values to guide model selection, quantization decisions, and potential TensorRT optimizations.

5.3.2 OCR Accuracy and Confidence Scoring

Once the plate is detected, OCR accuracy becomes the defining metric for successful license plate recognition. This includes two measurable components:

- Character Accuracy - the percentage of individual characters correctly recognized by the OCR engine.

- Sequence Accuracy - the percentage of entire license plates correctly recognized without any mistakes.

In addition to accuracy, the OCR model outputs a confidence score, which indicates how certain the model is in its predictions. We will use these confidence scores to implement automated thresholds. For example, flagging any results below a certain confidence as “needs review” in the database.

OCR performance is especially important because an incorrectly recognized plate can lead to mismatched weigh-in/weigh-out records. During testing, we plan to evaluate OCR performance using sample plate datasets and compare EasyOCR’s behavior under different environmental conditions and preprocessing settings.

5.3.3 System Latency (Capture → Database Storage)

System latency measures the total time required for the system to process a transaction from the moment the scale stabilizes to the moment the final record is inserted into the database.

This end-to-end latency includes:

1. Frame capture from the camera
2. YOLO detection inference
3. OCR text extraction
4. Cargo classification
5. Database insertion

Although each individual step may be fast, delays can accumulate and negatively affect operator workflow especially during peak hours at a weigh station. Our goal is to ensure that system latency is low enough that the process feels instantaneous from the operator’s perspective.

During testing, we will measure latency using timestamps at each stage of the pipeline and analyze how system performance changes when running on the Jetson Nano versus a desktop environment.

5.3.4 Stability and Reliability Measurements

Because the system is intended for continuous operation at real-world facilities, stability and reliability are just as important as model accuracy. This set of metrics assesses how well the system performs over long periods of time and under varying load conditions.

Key factors we plan to evaluate include:

- Uptime - how long the system can run without crashing or requiring manual restart.
- Frame Drop Rate - whether the vision pipeline maintains consistent frame capture.
- Error Frequency - rate of OCR failures, detection errors, or failed transactions.
- Thermal Stability - whether the Jetson Nano overheats during prolonged inference workloads.

These measurements help ensure that the system can operate unattended at night, in remote locations, or under high traffic without performance degradation.

Our reliability testing will involve running the full system for multiple continuous hours on the Jetson Nano, logging performance statistics, and identifying any bottlenecks or failure points.

6. Results and Conclusions

6.1. Characterization of Results

6.1.1 License Plate Recognition Outcomes

Since the implementation and testing phase of the license plate recognition (LPR) system has not yet begun, our current results are based on research, expected performance, and projected system behavior rather than empirical measurements. However, the design decisions we made allow us to outline clear expectations for the system's accuracy, reliability, and operational performance once testing is underway.

Based on benchmark studies of YOLOv8n and EasyOCR, we expect the LPR pipeline to perform well under controlled conditions where license plates are visible, well-lit, and captured at a consistent angle. YOLOv8n's detection accuracy on license plate datasets typically reaches high precision and recall for plate localization, while EasyOCR reliably handles alphanumeric text extraction when plates are not obstructed or excessively distorted. In our deployment environment, we anticipate achieving high character-level accuracy for most standard plate formats, especially after applying preprocessing steps such as cropping, contrast enhancement, and de-skewing.

We also expect that the system will encounter predictable challenges when dealing with real-world outdoor conditions. Motion blur, dirt-covered plates, nighttime lighting, overexposure, and angled views may reduce OCR performance or require additional preprocessing. Because we designed the pipeline with confidence scoring and fallback mechanisms, any low-confidence predictions will be flagged for manual review rather than automatically accepted. This approach ensures that early inaccuracies do not propagate into the database.

Overall, while true performance metrics cannot be reported yet, our research indicates that the YOLO + OCR architecture should deliver reliable, near-real-time LPR once integrated and optimized on the Jetson Nano. The next phase of work will focus on validating these expectations through systematic testing and refining the models based on real data from the sponsor's simulation and camera setup.

6.1.2 Cargo Classification Results

Like the LPR system, the cargo classification part of the project has not yet been fully implemented or tested, so we cannot show real accuracy numbers at this time. Instead, we describe what we expect the system to do based on our research into lightweight image-classification models and what the weigh-station environment will need from them. For the first version, we expect the model to work best on simple and easy-to-see materials such as mixed trash, construction debris, paper, or wood. Focusing on these clear categories helps us make sure the pipeline works before moving on to more confusing or visually similar materials. Since the model must run

smoothly on the Jetson Nano, the initial accuracy will likely be modest but still strong enough to prove that the system works in principle.

Most of the early challenges will come from real-world conditions. Cargo loads can look very different from one truck to another, and the camera angle will not always be perfect. Lighting changes throughout the day, shadows can cover parts of the cargo, and some items may be hidden behind others. Different materials also tend to blend together, making it harder for the model to tell them apart. Because of these challenges, early versions of the classifier will likely make mistakes, especially in messy or unclear scenes. As we begin collecting data from the sponsor's simulator and later from the real weigh-station camera, we will retrain the model using images that match the actual environment. This process should slowly improve the model and allow us to add more cargo types over time.

Even though we do not have numerical results yet, the system is built so that the classifier can be updated whenever needed without affecting the rest of the pipeline. Our first tests will focus on checking that the model can separate basic categories, spotting where it fails, and setting a simple baseline accuracy. After that, we can repeat the training process as more data arrives, gradually improving the model with each round.

As the project grows, one major improvement will come from collecting a large and diverse dataset. The more examples we have of different cargo types, lighting conditions, camera angles, and truck positions, the better the classifier will become. Once we gather enough real-world data, we can begin targeting difficult scenarios such as night-time operation, heavy rain, snow, fog, glare from the sun, and other conditions that are common at outdoor weigh stations.

We are not limited to only real footage, however. Data augmentation lets us artificially create many of these conditions during training. We can add effects like brightness changes, motion blur, fog, snow, shadows, or color shifts to existing images

to simulate new situations. This lets the model learn how to handle tricky cases without requiring thousands of real images for each weather type. Augmentation will not replace real data, but it expands the dataset and makes the model more adaptable.

Another stage of improvement involves using more than one camera instead of relying on a single view. A multi-camera setup could give the classifier different angles of the cargo as the truck approaches or sits on the scale. This would help in cases where one camera is blocked, the lighting is uneven, or part of the cargo is hidden from view. Multiple perspectives could also reduce confusion between materials that look similar from one angle but different from another. As the system becomes more advanced, combining information from several cameras will allow the model to build a more complete picture of the cargo and produce more reliable predictions.

Over time, combining real data, strong augmentation techniques, and multi-camera information will help the classifier handle more difficult cases and become more robust overall. As the dataset grows and we continue training in stages, the model should move closer to the level of accuracy needed for full deployment across different lighting conditions, weather patterns, and possibly multiple weigh-station sites.

6.2. Summary of Project

6.2.1 Lessons Learned

The Automated Weigh-In/Weigh-Out System aims to modernize and autonomize weight-station operations by integrating edge computing, computer vision, and real-time data processing. Built on the NVIDIA Jetson Nano, the system ultimately automates the capture of vehicle weight, license plate identification, and cargo classification.

Throughout Senior Design 1, our team focused primarily on establishing the technical foundation and validating the architecture needed for a complete implementation in Senior Design 2. We spent much of the early semester researching different

approaches, studying hardware limitations, and developing backend modules capable of communicating with simulated sensors. Despite delays in receiving key resources from the sponsor, we created a modular, well-structured system design that positions us for smoother progress in SD2.

Protocol Verification and Early System Assumptions

One of the most important lessons this semester came from an early misunderstanding about how the industrial truck scales communicate. At first, we assumed the scales used modern WebSocket-based communication. This led us to invest significant time researching async event-based architectures, Socket.IO libraries, and websocket client/server patterns. Once we finally received access to the scale simulator, we discovered the actual protocol was Telnet, an older, character-stream protocol that behaves nothing like WebSockets.

This mismatch forced us to pivot quickly, and rethink how data would be parsed and cached. More importantly, it emphasized how critical it is to confirm hardware specifications early. Even a small incorrect assumption can cause weeks of unnecessary research and slow down the design process. Moving forward, the team will prioritize direct confirmation from sponsors before committing to any architectural decision.

Access, Credentials, and Embedded Device Recovery

We encountered another major challenge when attempting to access the Jetson Nano. The device arrived without login credentials, which prevented us from installing packages, setting up the environment, or running any code. This pushed us into full recovery mode, requiring us to learn how to re-flash an embedded Linux device using its internal storage and NVIDIA's recovery tools. Although unplanned, this experience strengthened our understanding of embedded systems, EMMC management, and hardware-level troubleshooting.

This situation also underscored the importance of verifying device access, passwords, and provisioning details as early as possible. We now have a workflow in place for recovering the Jetson Nano if it becomes inaccessible again, which is extremely valuable considering that embedded devices can easily become corrupted during active development.

Data Dependency in Machine Learning Workflows

Another critical lesson involved dataset availability. Both the License Plate Recognition and Cargo Classification pipelines depend heavily on domain-specific images taken at the sponsor's facility. Because we did not receive this data during SD1, our ML progress was mostly limited to research, benchmarking, and architecture planning. This highlighted one of the biggest truths in machine learning: data must come before model development.

Without real footage of trucks entering and exiting the landfill, we cannot begin training or performing baseline evaluations. This constraint shaped much of our work and taught us to prioritize dataset collection early in SD2. We now have a clear plan for frame extraction, preprocessing, annotation, and augmentation as soon as the sponsor provides the video files.

6.2.2 Limitations and Future Improvements

Current Limitations

- **Simulation-Only Testing:** Because access to the physical scale hardware arrived late in the semester, and we still do not have access to the real PostgreSQL database instance, most testing relied on simulated weight streams and pre-recorded video. While the simulator allowed us to validate the basic flow of weight acquisition, it does not yet capture all the nuances of the real hardware.

- **Limited Cargo Dataset:** Our cargo classification model remains conceptual because we have not yet received training images of actual landfill loads. Without domain-specific data, we cannot run meaningful experiments or establish baseline accuracy.

Future Improvements

- **Firmware-Over-The-Air (FOTA):** A key long-term improvement is the ability to update field devices remotely. FOTA capabilities would allow the sponsor to securely deploy new ML models, patch software, and roll out enhancements across all weigh stations without requiring on-site technician visits. This is particularly important given the system's 24/7 deployment environments.
- **Driver Feedback System via GPIO:** Another enhancement involves using the Jetson Nano's GPIO pins to control a simple red/green LED system. This feedback mechanism would signal to drivers when the weight has stabilized, reducing operator workload and improving throughput at busy stations. This is a stretch goal, but it aligns well with the sponsor's interest in improving safety and efficiency.
- **Model Optimization for Embedded Deployment:** To ensure acceptable real-time performance on the Jetson Nano, we plan to apply model quantization (FP16 and INT8) and TensorRT acceleration. These optimizations will reduce inference time, improve throughput, and lower the device's thermal load—critical for long-duration operation.

6.2.3 Final System Evaluation

Although the system is still in the early stages of development, the work completed in Senior Design 1 has established a strong foundation for full implementation in Senior Design 2.

Infrastructure and Remote Collaboration: One of the most successful outcomes of this semester was the creation of a stable development workflow. Using Tailscale

VPN and SSH, our team can securely access the Jetson Nano from anywhere, allowing parallel development without requiring everyone to meet physically in the lab. This has significantly improved our efficiency and ensures that future testing on the Jetson will be smooth and repeatable.

Architectural Verification: We also completed initial architectural validation using the simulated scale data. The middleware responsible for caching live weight readings has been implemented and verified to function with the Telnet-based simulator. This confirms that our design for modular API endpoints, real-time weight polling, and transaction handling is sound, even though the system is not yet fully integrated.

By the end of Senior Design 1, the system has:

- a functional development environment
- confirmed system architecture
- a validated weight middleware layer
- a complete design plan for the computer vision components
- hardware access and recovery procedures fully resolved

While we still need real data and full integration to complete the system, the foundation built this semester positions us well for rapid development at the start of Senior Design 2. The next phase will focus heavily on implementation, dataset creation, model testing, hardware optimization, and end-to-end integration.

7. Administration

7.1. Budget and Financing

Below (Figure 14) is the table consisting of our current budget, with predicted hardware aspects for developing our Vehicle Weight System. As you can see the hardware makes up most of the cost analysis, as this will be the bulk of the budget. Currently It is a prediction for the future budget, with more hardware and software hosting aspects to be considered at a later date. The software side of the cost analysis will most likely be considered on a pay-as-you-go basis, scaling along with future sizing requirements and potential compute growth if need be. Figure 14 below shows our current predicted budget.

Figure 14: Budget

Part Name	Source	Quantity	Price
Jetson Orin Nano Development Kit	Sponsors	1	\$250
Camera	Amazon.com	2	\$50
		GRAND TOTAL:	\$350

The hardware requirements mainly consist of the Jetson Orin Nano and an estimated Camera cost, as we do not have access to a specific camera at this time. These will be the main packaging for this project in terms of cost analysis. Including potential future hardware additions, the hardware cost is projected to not exceed \$400, which is doable provided with recommendations from our sponsor.

For the software cost analysis, most of our software is open-source and free to use, which leads to a cheaper software end for this project. We predict that for future independent hosting, the cost should not exceed more than \$20 a month, given current software hosting standards.

The funding for this project will be provided by our sponsors, primarily regarding the hardware as the Jetson has already been provided and the camera still being decided on.

7.2. Agile and Scrum Artifacts

7.2.1 Project Management Overview

Throughout Senior Design 1, our team followed a lightweight Agile workflow grounded in Scrum principles. Because our project involves multiple subsystems (machine learning, embedded hardware, backend development, and integration) we adopted an iterative process that allowed us to adapt to evolving information and shifting sponsor timelines. Although external delays (hardware access, missing datasets, late simulator delivery) limited the amount of hands-on development we could complete, Scrum helped us stay organized, maintain progress, and continuously refine our plan as more resources became available.

Our Scrum structure consisted of three sprints during SD1, each lasting roughly two weeks, with dedicated sprint planning, sprint review, and regular standups. The goal for each sprint was to define a realistic set of tasks based on the resources currently available. This flexibility was essential, since many key dependencies (such as the Jetson Nano credentials, the scale simulator, and the camera footage) were outside our control and arrived late in the semester.

Sprint Planning

Sprint planning meetings occurred at the start of each sprint:

- Sprint 1 Planning: Mid-September
- Sprint 2 Planning: Early October
- Sprint 3 Planning: Early November

During planning, we reviewed the product backlog, clarified task ownership, and defined achievable goals for the upcoming two-week period. Because our sponsor's deliverables arrived gradually, the planning meetings also served as checkpoints to adjust scope and shift blocked tasks to later sprints. This iterative planning approach prevented major disruptions and allowed us to keep making progress even when larger components were unavailable.

Standups

We held weekly standups every Monday at the beginning of each week through Discord. These standups lasted approximately 15 minutes and followed a traditional three-question format:

- What did you complete last week?
- What will you work on this week?
- Are there any blockers?

The standups were effective, giving the team a consistent rhythm and helping us identify issues early such as delays waiting on sponsor resources or confusion about scale communication protocols. Since everyone had different schedules, Discord made it easy to meet regularly without needing on-campus availability.

Team Roles Under Scrum

Even though our team had defined technical roles, Scrum allowed us to collaborate fluidly when tasks crossed domains. The roles guided ownership but did not restrict contributions:

- Machine Learning & CV: License Plate Recognition, Cargo Classification
- Sensors & Hardware: Camera, Jetson Nano setup, scale simulator connectivity
- Backend & Database: PostgreSQL structure, API design
- Integration: Bringing all components together into a unified system

Scrum helped coordinate these different streams so each subsystem could progress independently when possible.

Outcome of Agile Use in SD1

Despite the significant resource delays during the first two sprints, using Scrum provided clear structure, steady progress, and predictable checkpoints. The Agile framework (standups, sprint planning, sprint reviews, and backlog grooming) kept the team synchronized and prepared us for more intensive development in Senior Design 2, when we finally expect to have all required data and hardware.

7.2.2 Product Backlog

The product backlog represents the full list of user stories and development tasks identified during Senior Design 1. These items were created and refined throughout the semester as we gained more information from the sponsor, recovered the Jetson Nano, and connected to the scale simulator. The product backlog continued to evolve as we refined our system architecture, validated assumptions, and adapted to delays in receiving hardware and data. It reflects not only the work we completed but also the foundational tasks required to prepare for full implementation in Senior Design 2.

Backlog items are grouped under the seven project epics:

- Epic 1 - Setup/Research
- Epic 2 - Machine Learning (AI/CV)
- Epic 3 - Sensors & Hardware
- Epic 4 - Backend & Database
- Epic 5 - Integration
- Epic 6 - Testing
- Epic 7 - Deployment & Documentation

Epic 1 - Setup/Research

- Research how to connect to Jetson Nano
- Research how to set up Jetson Nano
- Research Jetson Orin Nano capabilities
- Research backend technologies for server
- Research public datasets
- Research OCR models for license plate recognition
- Research existing cargo classification / license plate datasets
- Research whether cargo are single-material vs. mixed
- Research web socket integration (later corrected to Telnet protocol)
- Obtain access to hardware from sponsor

Epic 2 - Machine Learning (AI/CV)

- Decide on ML models for LPR
- Decide on ML models for cargo recognition
- Test YOLO + OCR on sample images

Epic 3 - Sensors & Hardware

- Set up and test camera connection
- Develop script to capture and display video feed
- Figure out how to connect a weight scale
- Make initial connection to weight scale simulator

Epic 4 - Backend & Database

- Create database backup
- Gain access to current database
- Connect backend to database
- Validate API/middleware design
- Confirm database backup and restore flow

Epic 5 - Integration

- Create Python script to parse JSON data from simulator
- Work on structuring logic for weight scale for each vehicle

Epic 6 - Testing

- Parse weight stream (via scale simulator)
- Verify LPR results on sample images (implicit in YOLO + OCR test)
- Validate API/middleware architecture

Epic 7 - Deployment & Documentation

- Work on final paper
- Organize system architecture documentation
- Prepare sprint summaries and design document sections

7.2.3 Sprint 1 - Research

Sprint 1 (9/29 - 10/13) focused entirely on foundational research and system discovery, since the team had not yet received key resources such as the Jetson Nano credentials, the scale simulator, or the sponsor's dataset. The tasks for this sprint centered on understanding the technologies, hardware, and workflows required for an automated weigh-in/weigh-out system.

Sprint Backlog (Tasks):

- Research how to connect to Jetson Nano
- Research whether cargo are single-material vs. mixed
- Research how to set up Jetson Nano
- Research web socket integration (later corrected to Telnet)
- Research Jetson Orin Nano capabilities
- Research public datasets
- Research existing cargo classification/license plate datasets
- Research OCR models for license plate recognition
- Research backend technologies for server
- Obtain access to hardware from sponsor

Sprint Summary: Sprint 1 focused entirely on foundational research since the team had not yet received hardware credentials, the scale simulator, or the dataset. Most tasks centered on understanding Jetson setup, reviewing backend options, evaluating machine-learning models, and studying weigh-station workflows. Early assumptions (such as believing the scale used WebSockets) guided initial research but were later corrected. The team also explored public datasets for cargo and license plate recognition, ultimately confirming that none matched the sponsor's environment. By the end of Sprint 1, the team had established a strong conceptual foundation and clarified the technical requirements needed for development in later sprints.

7.2.4 Sprint 2 - Environment Setup

Sprint 2 (10/13 - 10/27) centered on setting up the development environment and beginning early technical testing. With access to the Jetson Nano finally available, the team focused on configuring VPN/SSH, installing required software, and establishing database connectivity. This sprint marked the transition from research into hands-on setup work.

Sprint Backlog (Tasks):

- Recover Jetson Nano OS
- Set up Tailscale VPN and SSH
- Install dependencies and packages
- Set up PostgreSQL backup
- Test camera connection
- Capture and display video feed
- Decide LPR model architecture
- Connect to scale simulator
- Parse weight stream

Sprint Summary: Sprint 2 focused on turning the research from Sprint 1 into a functional, testable development environment. The team successfully restored access to the Jetson Nano, installed all required packages, and configured Tailscale VPN and SSH so everyone could safely access the device remotely. On the backend side, the team created a PostgreSQL database backup and began preparing the schema that would eventually store transaction data. We also started investigating the weight scale integration path, although full connection testing required tools that had not yet been delivered by the sponsor.

7.2.5 Sprint 3 - Semester Wrap-Up

Sprint 3 (11/10 - 11/24) concentrated on finalizing the system architecture, validating the scale communication pipeline, and completing documentation for Senior Design 1. By this point, the development environment was operational, allowing the team to shift from setup tasks toward refining logic, selecting final ML approaches, and preparing for implementation in Senior Design 2. Although the absence of the real dataset continued to limit machine-learning progress, the sprint allowed the team to solidify the project direction and ensure all core components were ready for integration next semester.

Sprint Backlog (Tasks):

- Create Python script to parse JSON data from simulator
- Decide on ML models for cargo recognition
- Work on final paper
- Work on structuring logic for weight scale for each vehicle
- Make initial connection to weight scale simulator
- Decide on ML models for LPR
- Gain access to current database
- Create database backup
- Research whether cargo are single-material vs. mixed

Sprint Summary: Sprint 3 focused on consolidating the work completed throughout the semester and preparing all components for Senior Design 2. The team successfully connected to the scale simulator, validated the Telnet-based communication flow, and began drafting the logic required to determine weigh-in vs. weigh-out direction. The machine learning team finalized model choices for both LPR and cargo classification, despite still lacking real training footage. The backend team secured database access and confirmed backup procedures, while documentation efforts (including diagrams, architectural sections, and the final design paper) became a major focus.

7.2.6 Senior Design 2 Sprint Plan

Senior Design 2 will shift from research and setup toward full implementation, training, testing, and deployment of the Automated Weigh-In/Weigh-Out System. The following sprint plan outlines the expected development flow for next semester. These timelines are approximate and will be refined once the sponsor provides the final dataset, hardware configuration details, and operational requirements.

Sprint 4 - Dataset & Pipeline Initialization (January)

Sprint 4 will begin with the most important dependency of the entire project: receiving the sponsor's real video dataset. The team will extract frames from the provided footage, clean and filter unusable images, and begin annotating both license plates and cargo categories. Preprocessing scripts for cropping, resizing, normalization, and dataset augmentation will be implemented during this sprint. We will also verify camera input on the Jetson Nano and ensure the scale simulator still operates correctly under the SD2 environment. This sprint essentially prepares all data and tooling needed to train the machine learning models in the following sprints.

Sprint 5 - Model Training & Integration (January/February)

Sprint 5 will focus entirely on training and integrating the License Plate Recognition and Cargo Detection pipeline. Using the cleaned dataset from Sprint 4, the team will train the models if needed. Once training is complete, we will deploy the model to the Jetson Nano and implement the full inference pipeline, ensuring that the system produces plate strings and cargo classifications reliably under real-world lighting and camera conditions. The final task of this sprint will be pushing results into the PostgreSQL database as part of the weigh-in/weigh-out transaction flow.

Sprint 6 - Full System Integration (March)

Sprint 6 will bring all major components together into a single, unified pipeline. This includes connecting the scale middleware, LPR pipeline, cargo classifier, backend API, and PostgreSQL database. The system will implement weigh-in and weigh-out logic, including direction detection and net-weight computation. This sprint will also

introduce exception handling, such as low OCR confidence or missing cargo predictions. The outcome of Sprint 6 is the first fully functional version of the system, capable of running the entire workflow end-to-end on the Jetson Nano.

Sprint 7 - Testing & Optimization (March)

Sprint 7 will focus on validating the system under realistic operating conditions. The team will run extensive tests using both the simulator and the sponsor's video segments to measure system latency, accuracy, reliability, and performance. Jetson-specific optimizations (such as INT8 or FP16 quantization and TensorRT deployment) will be applied to reduce inference time and stabilize performance. Error handling, logging, and reliability improvements will also be implemented. By the end of this sprint, the system should run smoothly, consistently, and efficiently.

Sprint 8 - Final Deployment & Documentation (April)

The final sprint will prepare the system for handoff and presentation. Tasks will include packaging the Jetson Nano software, finalizing API endpoints, writing installation and recovery documentation, demo, and project presentation. Additional real-world testing may be performed if new data is provided by the sponsor. By the end of Sprint 8, the project should be fully complete, demonstrated, and ready for deployment at the sponsor's site.

7.2.7 Setbacks and Resolutions

One of the biggest setbacks in the project has been the delays in communication with the sponsor, especially when it came to getting the key resources we needed early on. Several important items either arrived very late or were not provided when we needed them. These included the simulation tools for testing scale communication, access to the sponsor's backend database, the Jetson Nano hardware, and most importantly the training video dataset. Without these resources, we could not begin major components of the project, such as testing hardware connections, building a full data pipeline, or starting meaningful training of the machine-learning models.

While waiting, our team focused on everything we could work on. We researched potential technologies, reviewed documentation, and created small prototype programs so that we would be ready once the missing pieces arrived. We explored Python frameworks, learned how socket and Telnet communication works, studied the structure of streaming scale data, and experimented with lightweight ML models using PyTorch. Although this kept us productive, it could not replace the missing hardware or domain-specific dataset.

The missing dataset remains the largest obstacle. The cargo classification model must be trained on video captured from the sponsor's actual weigh-station camera, which has its own angle, lighting, and environmental conditions. Public datasets such as TrashNet are not viable replacements because they consist of close-up images of single objects, not overhead or angled views of mixed truck loads with clutter, shadows, and natural variation. A model trained on public images would almost certainly fail in real weigh-station conditions because the scale, distance, and visual structure of the scenes are entirely different.

If delays continue, we may temporarily use placeholder datasets or simplified images to test the end-to-end pipeline. This would allow us to verify the flow from the camera to the ML model, backend, and database. However, this would not represent real-world accuracy, and such a model would not be deployable. In the long term, the sponsor's dataset is essential. Without real footage that reflects the true camera viewpoint and weigh station environment, the ML models cannot learn the correct visual patterns, and progress on the cargo-classification and LPR components will remain limited.

8. Acknowledgments

We would like to acknowledge our project sponsor for providing access to key resources required for development, including the backend database, training video dataset, and the Jetson Nano hardware used for prototype testing. Their support also included access to simulation tools that allowed us to test scale communication and prototype system behavior without relying on a physical certified scale.

We also extend our appreciation to our faculty advisor for their guidance throughout the project, as well as the facilities that provided the computing resources and equipment needed to run machine-learning training, data processing, and hardware integration. These contributions were essential to enabling the design, testing, and evaluation of our system.

9. Bibliography

1. Butcher, T., Harshman, R., Konijnenburg, J., Lee, G. D., Williams, J., Warfield, L., Benham, E., Bowers, S., & Lippa, K. (2022). *Specifications, Tolerances, and Other Technical Requirements for Weighing and Measuring Devices (NIST HB 44-2023)*. National Institute of Standards and Technology, Gaithersburg, MD. <https://doi.org/10.6028/NIST.HB.44-2023> [NIST+2iswm.memberclicks.net+2](#)
2. National Institute of Standards and Technology. (n.d.). *NIST Handbook 44 – Current Edition*. Retrieved from <https://www.nist.gov/pml/owm/nist-handbook-44-current-edition> [NIST+1](#)
3. NVIDIA Corporation. (2019, December 17). *Jetson Nano Developer Kit User Guide* (DA_09402_003). Retrieved from https://developer.download.nvidia.com/embedded/L4T/r32-3-1_Release_v1.0/Jetson_Nano_Developer_Kit_User_Guide.pdf [NVIDIA Developer Download](#)
4. NVIDIA Corporation. (n.d.). *Jetson Nano 2 GB Developer Kit User Guide*. Retrieved from <https://developer.nvidia.com/embedded/learn/jetson-nano-2gb-devkit-user-guide> [NVIDIA Developer](#)
5. PyTorch Developers. (n.d.). *PyTorch Documentation* (v2.9). Retrieved from <https://docs.pytorch.org/> [PyTorch Docs+1](#)
6. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Chintala, S. (2019, December 3). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. arXiv. <https://arxiv.org/abs/1912.01703> [arXiv](#)
7. PostgreSQL Developers. (n.d.) *PostgreSQL*, Retrieved from www.postgresql.org/docs/
8. PHP Developers. (n.d), Retrieved from <https://www.php.net/>
9. Gosling, James; McGilton, Henry (May 1996). Retrieved from <https://www.oracle.com/technetwork/java/langenv-140151.html>